Efficient Polynomial System Solving via Dixon Resultants: Applications to AO Primitives

Haohai Suo[1,2,3] and Jiamin Cui[1,2,3(✉)]

[1] School of Cyber Science and Technology, Shandong University, Qingdao, Shandong, China
[2] State Key Laboratory of Cryptography and Digital Economy Security, Shandong University, Qingdao, 266237, Shandong, China
[3] Key Laboratory of Cryptologic Technology and Information Security, Ministry of Education, Shandong University, Jinan, China
haohai.suo@mail.sdu.edu.cn, cuijiamin@sdu.edu.cn

Abstract. Solving multivariate polynomial systems is a fundamental problem in cryptanalysis, with increasing relevance in algebraic attacks on arithmetization-oriented (AO) primitives. Current approaches primarily rely on Gröbner bases or the Sylvester resultant. However, Gröbner basis methods typically rely on FGLM to change the monomial order, which applies only to zero-dimensional ideals, whereas the Sylvester resultant eliminates only one variable at a time, limiting its flexibility in multivariate elimination.

We revisit the Dixon resultant as an efficient and flexible tool for eliminating several variables simultaneously. We derive refined upper bounds on the Dixon matrix size via lattice-path counting and analyze the complexity under several determinant computation models, yielding explicit complexity estimates. For well-determined systems, the Dixon resultant is a viable alternative to Gröbner basis methods; moreover, it is attractive for elimination in underdetermined systems, whereas Gröbner basis methods remain preferable for overdetermined ones.

We present an efficient open-source C implementation, `DRSolve`, with multiple determinant methods and a degree-aware submatrix selection strategy to mitigate the impact of extraneous factors. Experiments show that our implementation is competitive with the state-of-the-art Gröbner basis solvers Magma and msolve on randomly generated well-determined systems, with significant advantages in the low-variable/high-degree regime, while Magma and msolve remain preferable in the high-variable/low-degree regime.

Finally, we formulate three elimination strategies for polynomial systems arising from AO primitives: direct elimination, iterative elimination, and reduction-based hybrid elimination. We demonstrate these strategies on Poseidon, `Vision`, and `XHash12`, yielding complexity reductions in many cases.

Keywords: Dixon resultant, polynomial system solving, algebraic cryptanalysis, arithmetization-oriented primitives

1 Introduction

Solving multivariate polynomial systems over finite fields is a basic task in computational algebra. Three major viewpoints dominate the area: Gröbner basis methods [74,34,33], resultant-based elimination [29,63,55], and characteristic-set methods in the spirit of Wu's method [80]. In contemporary algebraic cryptanalysis, most attention is paid to Gröbner basis techniques [69,57], while resultant methods—especially the Dixon resultant as a compact multivariate elimination tool—remain much less explored in the algebraic cryptanalysis literature.

These problems are especially relevant for *arithmetization-oriented* (AO) primitives, whose round functions are deliberately designed to admit compact arithmetic representations in zero-knowledge (ZK) proof systems, MPC, and homomorphic settings. AO primitives differ substantially from classical binary-oriented designs. Instead of working over small binary fields, they are typically defined over large prime fields or extension fields, and their nonlinear layers are optimized for arithmetic circuits. Following this design philosophy, ZK-friendly AO primitives are commonly organized into three families (Types I–III):[4]

- *Type I (low degree):* MiMC [2], GMiMC [1], Poseidon [41], Poseidon2 [42], `Neptune` [44];
- *Type II (equivalence relations):* `Vision`/`Rescue` [4], `XHash12` [5], `Anemoi` [24], `Arion` [68], `Griffin` [38];
- *Type III (lookup tables):* `Reinforced Concrete` [39], `Monolith` [40], `Tip5` [75].

In all three settings, algebraic preimage and collision attacks against these primitives reduce to *constrained-input constrained-output* [18] (CICO) problems, which in turn amount to solving induced polynomial systems whose complexity dominates the security analysis.

Related work. Solving-based algebraic attacks have seen continuous refinement over the last several years. A major line of work [69,14,72,57,62,43,7] studies Gröbner basis attacks and, in particular, how to obtain better algebraic models or tighter upper bounds for their complexity on structured AO systems. In [57], the authors refined the complexity analysis by providing more precise estimates for the individual stages of the attack. However, in some cases the upper bounds obtained from such generic Gröbner basis analyses remain inaccurate. For example, the *FreeLunch* approach [14] uses dedicated weighted monomial orderings for which a given polynomial system is already a Gröbner basis, allowing for better complexity bounds.

Another line of work brings classical elimination theory into cryptanalysis. Yang et al. [81] introduced iterative pairwise resultant attacks based on the Sylvester resultant, eliminating one variable at a time from two equations. Bariant et al. [13] further improved this paradigm by exploiting structure inside the polynomial system and performing degree reductions during elimination.

[4] See the STAP Zoo taxonomy of ZK-friendly designs: https://stap-zoo.com/zk/.

These are important developments, but they remain rooted in iterated two-polynomial resultants rather than *multipolynomial* resultants, so the elimination order is comparatively rigid.

Dixon resultant. The Dixon resultant, introduced by A.L. Dixon in 1909 [32], offers a different determinant-based elimination mechanism: it can eliminate several variables simultaneously from a polynomial system and thus serves as a compact multipolynomial resultant in our setting. Compared with the Macaulay resultant [63], the Dixon construction often leads to substantially smaller matrices while preserving the essential elimination information. Kapur et al. [55] also explained how to handle the non-square situation via maximal-rank submatrices, which makes the method viable beyond the classical idealized setting. Tang and Feng [76] turned this construction into a solver by retaining one variable as a parameter, and Albrecht [3] implemented the same workflow against a block-cipher system. Our direct method follows this workflow, augmented with degree-aware submatrix selection and a fast polynomial-matrix determinant stage. Subsequently, numerous works [59,82,31,67,66,51,50] studied both the algebraic and algorithmic aspects of Dixon matrices.

For complexity analysis, the size of the Dixon matrix is the first key parameter. Kapur and Saxena [53] gave Newton-polytope and volume-based bounds for unmixed supports, while Chtcherba and Kapur developed multivariate methods for mixed supports [26] and exact size formulas for generic unmixed bivariate systems [27]. We derive an explicit ordered total-degree formula through lattice paths and a Hessenberg determinant, and prove generic attainment. For equal degrees, this extends the quadratic Catalan count of Tang and Feng [76]. A second key parameter is the cost of polynomial-matrix determinants themselves, as expansion by minors [37], Bareiss elimination [47], Kronecker substitution [61], Hermite normal form [58], evaluation-interpolation [47], and sparse interpolation [48] can lead to substantially different complexity estimates, and we accordingly develop a stage-wise complexity model that compares all of them.

Our contributions This paper studies Dixon resultants as a method for *solving polynomial systems*, with AO primitives as a representative application domain. Our main contributions are as follows.

1. We derive refined bounds on Dixon matrix size for mixed systems via lattice-path counting. For uniform-degree systems, the bound specializes to a Fuss–Catalan number [46,6]; for general mixed systems, we obtain an explicit Hessenberg determinant formula, which is attained by the monomial support of generic instances.
2. We develop a stage-wise complexity model for Dixon elimination by comparing multiple determinant computation strategies, yielding a more faithful description of polynomial-matrix behavior.
3. We implement an efficient C library `DRSolve`[5] supporting multiple determinant routines and a degree-aware submatrix heuristic. Experiments validate

[5] https://drsolve.github.io

Table 1. Notation used throughout the paper.

Symbol	Meaning
$\mathcal{F} = \{f_1, \ldots, f_n\}$	system with $\deg_{\boldsymbol{x}} f_i \le d_i$, $d_1 \le \cdots \le d_n$; an $(n; d)$-system if all $d_i = d$
$\boldsymbol{x}$, $\overline{\boldsymbol{x}}$, $\boldsymbol{a}$	$n-1$ eliminated variables, their dual variables, k retained parameters
C, $\widehat{C}$	cancellation matrix (1) and its refined form (3)
$\Delta_{\mathcal{F}}$, $\mathcal{M}$	Dixon polynomial and Dixon matrix, $\Delta_{\mathcal{F}} = X\mathcal{M}\overline{X}^T$
$D(\mathcal{F})$	Dixon matrix size of $\mathcal{F}$: the larger of its rows and columns
$\mathcal{M}'$, ρ	maximal-rank submatrix extracted from $\mathcal{M}$, of size $\rho = \operatorname{rank}\mathcal{M} \le D(\mathcal{F})$
$\mathcal{I}$, J, $d_{\mathcal{I}}$	$\langle\mathcal{F}\rangle$; elimination ideal $\mathcal{I} \cap \mathbb{K}[\boldsymbol{a}]$; degree of $\mathcal{I}$
$I_{d_1,\ldots,d_n}$, $C_n^{(d)}$	Iterated-Simplex polynomial (9); Fuss–Catalan number
ω	matrix multiplication exponent, $2 \le \omega < 3$

the proposed Dixon-matrix bounds and show competitiveness with Magma and msolve on well-determined random systems, with advantages in the low-variable/high-degree/large-field regime, while Magma and msolve remain superior in the high-variable/low-degree/small-field regime.

4. We organize Dixon-based elimination for AO-induced systems into three strategies: direct, iterative, and hybrid reduction. We illustrate them on Poseidon, `Vision`, and `XHash12`, respectively, providing concrete algebraic complexity estimates.

Outline. Section 2 reviews the Dixon construction and the determinant computation models used in the paper. Section 3 develops the refined Dixon-matrix bounds and the stepwise complexity analysis. Section 4 discusses implementation-oriented improvements. Section 5 presents the three application patterns on Poseidon, `Vision`, and `XHash12`.

2 Preliminaries

Notation. We denote fields with $\mathbb{K}$ and their algebraic closure by $\overline{\mathbb{K}}$. Throughout, $\mathcal{F} = \{f_1, \ldots, f_n\} \subset \mathbb{K}[\boldsymbol{a}][\boldsymbol{x}]$ is a system of n equations in $n-1+k$ variables, viewed as polynomials in the eliminated variables $\boldsymbol{x} = (x_1, \ldots, x_{n-1})$ with coefficients in $\mathbb{K}[\boldsymbol{a}]$, where $\boldsymbol{a} = (a_1, \ldots, a_k)$ are the retained parameters. The total degree of f is denoted by $\deg(f)$ and its degree in x_i by $\deg_{x_i}(f)$; $\langle\mathcal{G}\rangle$ denotes the ideal generated by a set $\mathcal{G}$ of polynomials. Table 1 collects the notation used throughout the paper.

2.1 Resultants

Resultants are classical tools in elimination theory used to determine whether a polynomial system has a common solution. We use the projection-operators viewpoint of [54].

Definition 1 (Resultant [54]). *Let $\boldsymbol{x} = (x_1, \ldots, x_{n-1})$ be the variables to eliminate and $\boldsymbol{a} = (a_1, \ldots, a_k)$ the retained parameters. For a system $\mathcal{F} = \{f_1, \ldots, f_n\} \subset \mathbb{K}[\boldsymbol{a}][\boldsymbol{x}]$, let $\mathcal{I} = \langle \mathcal{F} \rangle \subset \mathbb{K}[\boldsymbol{a}, \boldsymbol{x}]$ and $J := \mathcal{I} \cap \mathbb{K}[\boldsymbol{a}]$. Polynomials in J are called* projection operators *and vanish on the parameter values for which $\mathcal{F}$ has a common solution over $\overline{\mathbb{K}}$. If J is principal, its generator (up to scalar) is the* resultant *of $\mathcal{F}$.*

The Sylvester resultant [29] for two univariate polynomials of degrees d_1, d_2 produces a matrix of size $d_1 + d_2$; the Bézout–Cayley resultant [65,29] reduces this to $\max(d_1, d_2)$. The Macaulay resultant [63] generalizes this to several multivariate polynomials but can produce very large matrices. The Dixon construction [32] is a multivariate extension of the Bézout-style approach: it preserves the compactness of the Bézout–Cayley matrix while extending it from two to n polynomials, often yielding much smaller matrices than Macaulay.

Remark 1. For $k > 1$, J need not be principal, and determinant-based methods compute a specific nonzero element $R \in J$. The affine determinantal construction is defined as the determinant of a maximal-rank submatrix of the Dixon matrix $\mathcal{M}$ of Section 2.2 and vanishes on the projection of the affine solution set, but may include extraneous factors not essential for the projection. In this paper, we refer to this determinant R as the resultant in a slightly informal sense, rather than as a projection polynomial.

2.2 Dixon Resultant

The Dixon resultant method [32,55] eliminates several variables simultaneously via a determinant. The procedure has four steps [55,50].

Step 1: Compute the Dixon polynomial. Introduce $n-1$ auxiliary variables $\overline{x}_1, \ldots, \overline{x}_{n-1}$ and define the $n \times n$ cancellation matrix

$$C(\boldsymbol{x}, \overline{\boldsymbol{x}}) = \begin{bmatrix} f_1(x_1, x_2, \ldots, x_{n-1}) & \cdots & f_n(x_1, x_2, \ldots, x_{n-1}) \\ f_1(\overline{x}_1, x_2, \ldots, x_{n-1}) & \cdots & f_n(\overline{x}_1, x_2, \ldots, x_{n-1}) \\ f_1(\overline{x}_1, \overline{x}_2, \ldots, x_{n-1}) & \cdots & f_n(\overline{x}_1, \overline{x}_2, \ldots, x_{n-1}) \\ \vdots & \ddots & \vdots \\ f_1(\overline{x}_1, \overline{x}_2, \ldots, \overline{x}_{n-1}) & \cdots & f_n(\overline{x}_1, \overline{x}_2, \ldots, \overline{x}_{n-1}) \end{bmatrix}. \tag{1}$$

The Dixon polynomial is

$$\Delta(\boldsymbol{x}, \overline{\boldsymbol{x}}) = \frac{\det(C(\boldsymbol{x}, \overline{\boldsymbol{x}}))}{\prod_{i=1}^{n-1}(x_i - \overline{x}_i)}. \tag{2}$$

Instead of performing an explicit symbolic division, we use the refined construction from [51]

$$\mathrm{Row}(\widehat{C}_1) = \mathrm{Row}(C_1), \qquad \mathrm{Row}(\widehat{C}_{i+1}) = \frac{\mathrm{Row}(C_{i+1}) - \mathrm{Row}(C_i)}{x_i - \overline{x}_i}, \tag{3}$$

for $i = 1, \ldots, n-1$. Each numerator is divisible by $x_i - \overline{x}_i$ in $\mathbb{K}[\boldsymbol{x}, \overline{\boldsymbol{x}}, \boldsymbol{a}]$, so the quotient remains polynomial; the Dixon polynomial is $\Delta(\boldsymbol{x}, \overline{\boldsymbol{x}}) = \det(\widehat{C})$. This row-wise difference-quotient construction is used throughout the paper.

Step 2: Construct the Dixon matrix. The Dixon polynomial admits a bilinear form $\Delta(\boldsymbol{x}, \overline{\boldsymbol{x}}) = X\mathcal{M}\overline{X}^T$, where X and $\overline{X}$ list monomials in $\boldsymbol{x}$ and $\overline{\boldsymbol{x}}$, respectively. The coefficient matrix $\mathcal{M}$ is the **Dixon matrix**: its rows are indexed by the $\boldsymbol{x}$-monomials and its columns by the $\overline{\boldsymbol{x}}$-monomials occurring in Δ, so it is in general rectangular and rank-deficient. Alternatively, $\mathcal{M}$ can be constructed directly via the recursive block method of Zhao and Fu [82], particularly relevant when few variables are eliminated and degrees are high [66].

Step 3: Extract a maximal-rank submatrix. In practice, the Dixon matrix $\mathcal{M}$ is often rectangular or rank-deficient, so its full determinant is unavailable or identically zero. Following the extension of Dixon elimination to singular matrices by KSY [55], we extract a maximal-rank square submatrix $\mathcal{M}'$ of size $\operatorname{rank}\mathcal{M}$ whose determinant is nonzero. Following [51,50], such a submatrix is typically identified efficiently after evaluating $\mathcal{M}$ at a generic point of $\mathbb{K}$.

Remark 2. The validity of using $\det(\mathcal{M}')$ relies on the KSY condition for singular Dixon matrices. It may theoretically fail, but such instances are extremely rare in practice [55] and it was satisfied for every instance reported in this paper.

Step 4: Compute the Dixon resultant. Once $\mathcal{M}'$ is obtained, $\det(\mathcal{M}')$ is the Dixon resultant in $\mathbb{K}[\boldsymbol{a}]$. Under the KSY condition, it remains a valid projection polynomial for a singular Dixon matrix: it vanishes whenever the original system has a common solution in the eliminated variables. In general, $\det(\mathcal{M}')$ may contain extraneous factors.

Size of the Dixon matrix. The size of the Dixon matrix is a key parameter in the complexity analysis. A classical variable-wise upper bound [55,66] is

$$B_{var} = (n-1)! \prod_{i=1}^{n-1} m_i, \tag{4}$$

where $m_i = \max_{1 \le j \le n} \deg_{x_i}(f_j)$ is the maximum degree in x_i over all polynomials in $\mathcal{F}$. Kapur and Saxena [53, Theorem 4.3] give a volume-based asymptotic estimate for multi-homogeneous systems. In the equal-total-degree setting, their

volume expression is $(dn)^{n-1}/n!$. We use its integer part as the comparison bound

$$B_{MH} = \left\lfloor \frac{(dn)^{n-1}}{n!} \right\rfloor . \tag{5}$$

Section 3 derives exact generic support formulas for the total-degree setting and independently verifies the explicit inequality $D(\mathcal{F}) \leq B_{MH}$ from the Fuss–Catalan formula.

Influence of extraneous factors. The determinantal output may contain factors unrelated to the affine solution set.

Definition 2 (Extraneous factor). *Let Z be the Zariski closure of the projection of $V(\mathcal{I})$ onto the retained parameters. An irreducible factor e of the determinantal elimination polynomial is an* ***extraneous factor*** *if $V(e) \not\subseteq Z$.*

In the absence of such factors and of excess multiplicities, the total degree of the eliminant is bounded by $d_{\mathcal{I}}$, both when $k = 1$ and when $k \geq 2$; in either case it is bounded by the Bézout bound:

Theorem 1 (Bézout bound [45,36]). *Let $\mathcal{I} = \langle f_1, \ldots, f_m \rangle \subset \mathbb{K}[x_1, \ldots, x_N]$ with $\deg(f_i) = d_i$, and let $d_{\mathcal{I}}$ denote the degree of $\mathcal{I}$, i.e. the number of solutions counted with multiplicity when $\mathcal{I}$ is zero-dimensional, and the degree of the affine variety $V(\mathcal{I})$ in general. Then*

$$d_{\mathcal{I}} \leq \prod_{i=1}^{m} d_i.$$

In the presence of extraneous factors, the degree of the eliminant may exceed the Bézout bound of the underlying system, both inflating computational costs and introducing spurious components unrelated to the actual solution set.

2.3 Polynomial Matrix Determinant Computation

We summarize the determinant complexity models used in this paper; full details are in Appendix I.

Univariate determinants. For $P(t) \in \mathbb{K}[t]^{m \times m}$, the Hermite normal form (HNF) method of [58] has deterministic cost $T^{\mathrm{HNF}} = \widetilde{O}(m^{\omega} \lceil \delta \rceil)$, where δ is the average column degree.

Multivariate determinants. For $P(\boldsymbol{y}) \in \mathbb{K}[y_1, \ldots, y_v]^{m \times m}$, no single method is universally best: different algorithms can lead to substantially different complexity estimates depending on the matrix size, the number of variables, and the sparsity structure of the entries. We therefore consider several alternatives. Besides direct symbolic expansion by minors [47,37] and fraction-free Bareiss

Table 2. Determinant computation models used in this paper. All complexities are in field operations over $\mathbb{F}_q$.

Method	Time	Space
Expansion [37,47]	$\widetilde{O}(m2^m\mu U)$	$\widetilde{O}(\binom{m}{m/2}U)$
Bareiss [12,37]	$\widetilde{O}(m^3U^2)$	$\widetilde{O}(m^2U)$
Kronecker [61]	$\widetilde{O}(m^\omega\delta)$	$\widetilde{O}(m\delta)$
Interpolation [47]	$\widetilde{O}(N(L+vB))$	$\widetilde{O}(N)$
Sparse [48][6]	$\widetilde{O}(LS+S\log q)$	$\widetilde{O}(S)$

elimination [12], one may reduce to the univariate case via Kronecker substitution [61] and apply HNF, apply evaluation-interpolation on a tensor grid [47], or use sparse interpolation [48]. The corresponding time and space costs are summarized in Table 2.

In the table notation, b_i bounds $\deg_{y_i}\det(P)$ and fixes the Kronecker substitution base (chosen to avoid overflow in the encoded determinant); $B := \max_i b_i$; $N := \prod_{i=1}^{v}(b_i+1)$ is the tensor-grid size used by interpolation; S bounds the number of nonzero terms of $\det(P)$; U bounds the term count of intermediate minors; L is the cost of one determinant probe; and $\mu \le \binom{v+d}{d}$ bounds the dense monomial count of an entry of total degree at most d. All complexities are reported in field operations over $\mathbb{F}_q$; derivations are given in Appendix I.

2.4 Security of AO Primitives

The sponge construction [19] builds hash functions from permutations. The relevant task for algebraic attacks is the constrained-input constrained-output [18] (CICO) problem.

Definition 3 (CICO problem). *For a permutation $F\colon \mathbb{F}_q^t \to \mathbb{F}_q^t$, the CICO-$k$ problem is to find $x \in \mathbb{F}_q^{t-k} \times \{0\}^k$ and $y \in \mathbb{F}_q^{t-k} \times \{0\}^k$ with $F(x) = y$.*

Brute force requires $\sim q^k$ permutation calls; substantially faster algebraic solutions may translate into preimage or collision attacks in the sponge setting.

3 Improved Complexity Estimates for the Dixon Resultant

Newton-polytope bounds for unmixed systems [53] and multivariate support methods for mixed systems [26] provide the starting point for our analysis. We derive an explicit lattice-path bound from ordered total-degree bounds and prove that it is attained for generic systems in the corresponding coefficient space.

[6] The sparse model also requires $\mathrm{char}(\mathbb{F}_q) \ge B$ (Huang [48, Theorem 1]); it therefore applies to large-prime fields but not to extension fields of the form $\mathbb{F}_{2^\ell}$.

Throughout this section, $D(\mathcal{F})$ is the size of the Dixon matrix $\mathcal{M}$ of $\mathcal{F}$ (Table 1); it upper-bounds $\rho = \operatorname{rank} \mathcal{M}$ but is in general strictly larger (Remark 5). For generic systems, and under the characteristic hypothesis of Lemma 4, the two monomial counts coincide and $\mathcal{M}$ is square, which is the situation described by the formulas below. A system of n polynomials with $\deg_{\boldsymbol{x}}(f_i) \le d_i$ is called a $(d_1, \dots, d_n)$-system, and an $(n; d)$-system when $d_1 = \cdots = d_n = d$, following [53].

3.1 A Refined Bound on the Size of the Dixon Matrix

We bound the Dixon matrix size for general $(d_1, \dots, d_n)$-systems via a lattice path count. Background is given in Appendix A.

Theorem 2 (Lattice Path Bound). *For a $(d_1, d_2, \dots, d_n)$-system $\mathcal{F}$ with $d_1 \le d_2 \le \cdots \le d_n$, let*

$$a_i := \sum_{k=1}^{i} (d_{n+1-k} - 1), \qquad i = 1, 2, \dots, n-1.$$

Then $D(\mathcal{F})$ is bounded above by the number of lattice paths from $(0, 0)$ to $(n-1, a_{n-1})$ such that for every $i = 1, 2, \dots, n-1$, the height y_i of the i-th horizontal step is at most a_i. Moreover, if $\operatorname{char}(\mathbb{K})$ is zero or sufficiently large, this bound is attained for generic polynomial systems.

Remark 3. The bound and the generic matrix size are independent of d_1. For nested total-degree supports, the support relations in [26, Theorems 3.1 and 5.1] imply that replacing the smallest-support equation by 1 preserves generic support. The proof below gives an explicit system attaining the bound.

This lattice path counting problem admits precise closed-form solutions through classical results in enumerative combinatorics.

Lemma 1. *Let $a = (a_1, a_2, \dots, a_{n-1})$ be an integer sequence with $0 \le a_1 \le a_2 \le \cdots \le a_{n-1}$. Consider lattice paths from $(0, 0)$ to $(n-1, a_{n-1})$.*

(1) The total number of lattice paths without any restriction equals

$$\binom{(n-1) + a_{n-1}}{n-1}.$$

(2) If $a_i = i(d-1)$ for some constant d and all $i = 1, \dots, n-1$, then the number of lattice paths such that the height y_i of the i-th horizontal step is at most a_i equals the Fuss–Catalan number with parameters (n, d)

$$C_n^{(d)} := \frac{1}{(d-1)n + 1} \binom{dn}{n}.$$

(3) The number of lattice paths such that for every $i = 1, 2, \dots, n-1$ the height y_i of the i-th horizontal step is at most a_i is given by the determinant

$$\det_{1 \le i, j \le n-1} \left(\binom{a_i + 1}{j - i + 1} \right).$$

Proof. Parts (1) and (2) are classical results in enumerative combinatorics; see [46,6]. Part (3) is a special case of Theorem 10.7.1 in [21] with $b_j = 0$ for all j. □

By combining Theorem 2 and Lemma 1, we immediately obtain the following bounds.

Corollary 1. *For a $(d_1, d_2, \ldots, d_n)$-system $\mathcal{F}$ with $d_1 \le d_2 \le \cdots \le d_n$, the following three bounds on the Dixon matrix size hold:*

(1) ***Unrestricted Bound:***

$$D(\mathcal{F}) \le \binom{\sum_{i=2}^{n}(d_i - 1) + n - 1}{n-1} = \binom{\sum_{i=2}^{n} d_i}{n-1}. \tag{6}$$

This bound is obtained by counting all lattice paths from $(0,0)$ to $(n-1, \sum_{i=2}^{n}(d_i - 1))$ without boundary constraints.

(2) ***Fuss–Catalan Bound (for $(n;d)$-systems):*** *Assume $d_1 = \cdots = d_n = d$.*

$$D(\mathcal{F}) \le C_n^{(d)} = \frac{1}{(d-1)n+1}\binom{dn}{n}. \tag{7}$$

The bound is attained for generic systems by Theorem 2 and Lemma 1(2), under the characteristic hypothesis of Lemma 4.

(3) ***Hessenberg Bound:***

$$D(\mathcal{F}) \le \det_{1 \le i,j \le n-1} \left(\binom{\sum_{k=1}^{i}(d_{n+1-k} - 1) + 1}{j - i + 1} \right). \tag{8}$$

The bound is attained for generic systems by Theorem 2 and Lemma 1(3), under the same hypothesis.

Remark 4. The matrix in (8) is a Hessenberg matrix, hence its determinant can be computed in $\mathcal{O}(n^2)$ using specialized algorithms [25] or [64, Theorem 1.9].

Outline of proof of Theorem 2. We describe the support of an Iterated-Simplex Polynomial by linear inequalities (Lemma 2), bound the Dixon matrix size by its monomial count (Lemma 4), and count these monomials using lattice paths (Lemma 5).

Definition 4 (Iterated-Simplex Polynomial). *Let n be a positive integer and $(d_1, d_2, \ldots, d_n)$ be a sequence of nonnegative integers. The Iterated-Simplex polynomial is defined as*

$$I_{d_1,d_2,\ldots,d_n}(x_1, \ldots, x_n) := \prod_{k=1}^{n} \left(\sum_{\boldsymbol{\alpha} \in \mathbb{Z}_{\ge 0}^{k},\ \alpha_1 + \cdots + \alpha_k \le d_k} x_1^{\alpha_1} \cdots x_k^{\alpha_k} \right), \tag{9}$$

where the k-th factor represents the complete sum of monomials in the first k variables with total degree at most d_k.

Now we characterize the support of the Iterated-Simplex polynomial in terms of linear inequalities.

Lemma 2. *The support of the Iterated-Simplex Polynomial $I_{d_1,\dots,d_n}$, taken over $\mathbb{Z}$, is given by*

$$\mathrm{Supp}(I_{d_1,\dots,d_n}) = \Big\{ \boldsymbol{\alpha} \in \mathbb{Z}^n_{\geq 0} \ : \ \sum_{l=n-j+1}^{n} \alpha_l \leq \sum_{i=1}^{j} d_{n-i+1} \text{ for } j = 1,\dots,n \Big\}. \tag{10}$$

The proof is given in Appendix B.

To compare different degree sequences, we establish a monotonicity property with respect to the trailing partial sums; it covers both reorderings and entry-wise decreases of the degrees.

Lemma 3. *Let $(d_1,\dots,d_n)$ and $(d'_1,\dots,d'_n)$ be sequences of nonnegative integers satisfying $\sum_{j=1}^{k} d'_{n-j+1} \leq \sum_{j=1}^{k} d_{n-j+1}$ for every $k = 1,\dots,n$. Then*

$$\mathrm{Supp}(I_{d'_1,\dots,d'_n}) \subseteq \mathrm{Supp}(I_{d_1,\dots,d_n}).$$

In particular, if $d_1 \leq \cdots \leq d_n$, the hypothesis holds for any permutation of a sequence obtained by decreasing some of the d_i.

Proof. Let $\boldsymbol{\alpha} \in \mathrm{Supp}(I_{d'_1,\dots,d'_n})$. By Lemma 2, for every $k = 1,\dots,n$,

$$\sum_{\ell=n-k+1}^{n} \alpha_\ell \leq \sum_{j=1}^{k} d'_{n-j+1} \leq \sum_{j=1}^{k} d_{n-j+1}.$$

These are precisely the inequalities defining $\mathrm{Supp}(I_{d_1,\dots,d_n})$, so the inclusion follows. If the degrees are first decreased and then reordered, any k resulting entries sum to at most the k largest original entries. Under the sorted ordering, their sum is $d_{n-k+1} + \cdots + d_n$, which verifies the hypothesis. □

Connection to the Dixon Matrix Size. We now connect the Dixon matrix size to the Iterated-Simplex Polynomial.

Lemma 4. *Let $\mathcal{F} = \{f_1,\dots,f_n\} \subset \mathbb{K}[x_1,\dots,x_{n-1}]$ have total degrees at most $d_1 \leq \cdots \leq d_n$. Then the Dixon matrix size is bounded above by the number of monomials in the Iterated-Simplex Polynomial $I_{d_2-1,\dots,d_n-1}$, taken over $\mathbb{Z}$. This bound is attained for generic systems if* $\mathrm{char}(\mathbb{K})$ *is zero or sufficiently large.*

Proof. Let $S = \mathrm{Supp}(I_{d_2-1,\dots,d_n-1})$ over $\mathbb{Z}$, and write $N^* = |S|$. In the bilinear form $\Delta_{\mathcal{F}} = X\mathcal{M}\overline{X}^T$, the column and row labels are the monomials in $\bar{\boldsymbol{x}}$ and $\boldsymbol{x}$, respectively.

Upper bound. We use the support argument of [53, Lemma 3.3 and Theorem 3.4] with the refined matrix (3). Its first row has no dual variables. For $i \geq 2$, the entry $\widehat{C}_{i,j}$ involves only $\bar{x}_1,\dots,\bar{x}_{i-1}$ and has dual degree at most $d_j - 1$,

since a difference quotient lowers the degree by one. Thus each determinant term satisfies

$$\mathrm{Supp}_{\bar{\boldsymbol{x}}}\left(\prod_{i=1}^{n}\widehat{C}_{i,\sigma(i)}\right)\subseteq \mathrm{Supp}(I_{d_{\sigma(2)}-1,\ldots,d_{\sigma(n)}-1})\subseteq S.$$

Indeed, in the main-diagonal term the last $n-1$ rows have the sorted degree bounds $d_2-1,\ldots,d_n-1$. For any other permutation σ, a monomial with dual exponent vector $\boldsymbol{\alpha}$ satisfies, for $1\le k<n$,

$$\sum_{\ell=n-k}^{n-1}\alpha_\ell \le \sum_{i=n-k+1}^{n}(d_{\sigma(i)}-1)\le \sum_{j=n-k+1}^{n}(d_j-1).$$

The second inequality bounds the selected degrees by the k largest ones. Lemma 3 therefore places every determinant term in S. Summing these terms can remove monomials through cancellation, but cannot introduce monomials outside S. Hence $\Delta_{\mathcal{F}}$ has at most N^* dual monomials.

For the row count, use the variable-reversal symmetry of [26, proof of Proposition 5.1]. Let $\mathcal{F}^{\mathrm{rev}}$ replace each x_i by x_{n-i}, and let τ interchange x_i with $\bar{x}_{n-i}$. Reversing the rows of the cancellation matrix gives

$$\tau(\Delta_{\mathcal{F}})=\pm\Delta_{\mathcal{F}^{\mathrm{rev}}}. \tag{11}$$

Since $\mathcal{F}^{\mathrm{rev}}$ has the same degree bounds, its dual-monomial bound gives the required N^* bound for the original monomials of $\Delta_{\mathcal{F}}$.

Attaining the bound. Consider the nested system

$$f_1^*=1,\qquad f_i^*=(x_1+\cdots+x_{i-1})^{d_i},\quad 2\le i\le n.$$

Taking $f_1^*=1$ is allowed because the prescribed degrees are upper bounds. For $j<i$, the polynomial f_j^* does not involve x_{i-1}. Consequently, the two evaluations in the numerator of the row-i difference quotient agree, giving $\widehat{C}_{i,j}=0$. Thus

$$\widehat{C}=\begin{bmatrix}1 & f_2^* & \cdots & f_n^*\\ 0 & \widehat{C}_{2,2} & \cdots & \widehat{C}_{2,n}\\ \vdots & \ddots & \ddots & \vdots\\ 0 & \cdots & 0 & \widehat{C}_{n,n}\end{bmatrix},\qquad \Delta_{\mathcal{F}^*}=\prod_{i=2}^{n}\widehat{C}_{i,i}.$$

Put $s_j=\bar{x}_1+\cdots+\bar{x}_j$, with $s_0=0$. The diagonal entries are explicit difference quotients:

$$\begin{aligned}\widehat{C}_{i,i}&=\frac{s_{i-1}^{d_i}-(s_{i-2}+x_{i-1})^{d_i}}{x_{i-1}-\bar{x}_{i-1}}\\ &=-\sum_{t=0}^{d_i-1}(s_{i-2}+x_{i-1})^t s_{i-1}^{d_i-1-t}.\end{aligned}$$

Here the second equality is the difference-of-powers identity. Setting all $x_i = 1$ gives

$$\Delta_{\mathcal{F}^*}(\mathbf{1}, \bar{\boldsymbol{x}}) = (-1)^{n-1} \prod_{i=2}^{n} \left(\sum_{t=0}^{d_i-1} (1 + s_{i-2})^t s_{i-1}^{d_i-1-t} \right).$$

Each factor has degree at most $d_i - 1$ and involves only the first $i - 1$ dual variables. Conversely, fix a monomial of degree $q \leq d_i - 1$ in these variables. Taking $t = d_i - 1 - q$ and the constant term of $(1 + s_{i-2})^t$ leaves s_{i-1}^q, whose multinomial expansion contains that monomial. The factor therefore has exactly the complete simplex support of degree $d_i - 1$. All its coefficients are positive integers, so their products cannot cancel in characteristic zero or sufficiently large characteristic (Remark 5). Multiplying the factors gives precisely the support S of $I_{d_2-1,\ldots,d_n-1}$. Since specializing $\boldsymbol{x}$ can only remove dual monomials, the unspecialized Dixon polynomial also attains the dual-monomial bound.

Generic attainment. Each coefficient just exhibited in $\Delta_{\mathcal{F}}(\mathbf{1}, \bar{\boldsymbol{x}})$ is a polynomial in the input coefficients, nonzero at $\mathcal{F}^*$. The coefficient space over $\overline{\mathbb{K}}$ is an affine space and hence irreducible. The simultaneous nonvanishing of these finitely many coefficient polynomials defines a nonempty open set U, which is therefore dense; see [29, Chapter 3, §5, Definition (5.6), p. 115]. Reversing the variable order permutes the input coefficients, so the corresponding set U^{rev} is also dense and open. On the nonempty intersection $U \cap U^{\mathrm{rev}}$, equation (11) gives N^* monomials on both sides. Thus the matrix is generically square of size N^*. □

Remark 5. The size $D(\mathcal{F})$ counts row and column monomials and upper-bounds $\rho = \operatorname{rank} \mathcal{M}$; generic attainment of this size does not assert full rank. The upper bound holds over any field. A sufficient characteristic condition for attainment is

$$\operatorname{char}(\mathbb{K}) = 0 \quad \text{or} \quad \operatorname{char}(\mathbb{K}) > \prod_{i=2}^{n} d_i (i-1)^{d_i-1},$$

because the product on the right is the sum of the absolute values of the integer coefficients in the specialized product above, and hence bounds each coefficient.

From Polyhedral Constraints to Lattice Paths We now establish a bijection between lattice points satisfying the polyhedral constraints and lattice paths below a general boundary.

Lemma 5. *Let $a_1 \leq a_2 \leq \cdots \leq a_n$ be nonnegative integers. The number of lattice points $(\alpha_1, \ldots, \alpha_n) \in \mathbb{Z}^n_{\geq 0}$ satisfying*

$$\alpha_1 + \cdots + \alpha_k \leq a_k, \quad k = 1, \ldots, n,$$

is equal to the number of lattice paths from $(0, 0)$ to (n, a_n) such that for every $i = 1, 2, \ldots, n$ the height y_i of the i-th horizontal step is at most a_i.

Proof. Given a lattice point, set $\alpha_{n+1} = a_n - \sum_{i=1}^{n} \alpha_i$ and form the path

$$U^{\alpha_1} R U^{\alpha_2} R \cdots U^{\alpha_n} R U^{\alpha_{n+1}},$$

where U and R denote upward and rightward steps. This path has exactly n horizontal steps, and its k-th horizontal step has height $\alpha_1 + \cdots + \alpha_k \le a_k$. The final α_{n+1} upward steps bring it to (n, a_n); they are allowed because the height constraint applies only to horizontal steps.

Conversely, given an admissible path, let α_i count the upward steps after the $(i-1)$-st horizontal step and before the i-th, taking the starting point when $i = 1$. The height of the k-th horizontal step is then $\alpha_1 + \cdots + \alpha_k$, so the required inequalities hold. The endpoint uniquely fixes the remaining upward steps. The two constructions are inverse to each other, giving the desired bijection. □

To prove Theorem 2, it remains by Lemma 4 to count the monomials of $I_{d_2-1,\ldots,d_n-1}$. Applying Lemma 2 in $n-1$ variables, their exponent vectors satisfy

$$\sum_{\ell=n-j}^{n-1} \alpha_\ell \le \sum_{i=1}^{j} (d_{n-i+1} - 1), \qquad 1 \le j < n.$$

Reversing the coordinates via $\beta_i = \alpha_{n-i}$ turns these into the prefix constraints $\beta_1 + \cdots + \beta_j \le a_j$. By Lemma 5, the resulting nonnegative lattice points correspond exactly to paths from $(0, 0)$ to $(n-1, a_{n-1})$ whose j-th horizontal step has height at most a_j. This is the boundary in the theorem. Generic attainment follows from Lemma 4. □

Comparison with Existing Bounds Our bound refines the equal-degree volume estimate and extends the dense bivariate and uniform quadratic cases to arbitrary dimensions and unequal degree bounds. For $n = 3$ and equal degrees, our formula gives $C_3^{(d)} = d(3d-1)/2$, which also follows from [27, Theorem 4.4]. For $d = 2$, it recovers the Catalan count of [76]. For general n, comparison with the volume estimate of [53, Theorem 4.3] gives

$$C_n^{(d)} = \frac{1}{n!} \prod_{j=0}^{n-2} (nd - j) \le \frac{(nd)^{n-1}}{n!},$$

which also verifies the integer volume bound B_{MH} in (5). Numerical comparisons and support-size experiments are given in Appendix C.

3.2 Complexity Estimates via Our Refined Bound

For clarity of exposition, we focus on $(n; d)$-systems where all polynomials have total degree d. Consider $\mathcal{F} = \{f_1, \ldots, f_n\} \subset \mathbb{K}[\boldsymbol{x}, \boldsymbol{a}]$ consisting of n equations in $n - 1 + k$ variables, where $\boldsymbol{x} = (x_1, \ldots, x_{n-1})$ are the eliminated variables and $\boldsymbol{a}$ the retained parameters. By Corollary 1(2), the generic Dixon matrix size is

$$C_n^{(d)} = \frac{1}{n(d-1)+1} \binom{nd}{n}.$$

Since $\rho \le D(\mathcal{F})$, the quantity $C_n^{(d)}$ also upper-bounds the size of the extracted submatrix $\mathcal{M}'$, so the estimates below are upper bounds on the actual cost.

We give a stage-by-stage outline of the relevant model choices below; here we consider the determinant methods with the lowest theoretical complexity or the fastest experimental performance. The detailed parameter derivations, complexity formulas for all five determinant models, and the full Step 1 vs Step 4 comparison are deferred to Appendix H.

Step 1: Compute the cancellation determinant. This step computes det of the $n \times n$ cancellation matrix (1), whose entries are polynomials in $\ell = 2n+k-2$ variables (the $n-1$ original variables, $n-1$ dual variables, and k parameters), with total parameter-degree bounded by $B_1 := nd$. This is the *many-variable, small-matrix* regime, where expansion by minors and sparse interpolation are most competitive. The abstract parameters governing these models admit the following explicit expressions in $(n, d, k, C_n^{(d)})$ (full derivation in Appendix H):

$$\mu_1 \le \binom{n-1+k+d}{n-1+k},\ S_1 \le \left(C_n^{(d)}\right)^2 \binom{nd+k}{k},\ L_1 = \widetilde{O}\left(n^2 \binom{n-1+k+d}{n-1+k}\right), \tag{12}$$

where μ_1 counts the monomials of one dense polynomial of total degree d in $n-1+k$ variables, the factor $\left(C_n^{(d)}\right)^2$ in S_1 comes from the Dixon bilinear support and $\binom{nd+k}{k}$ from the parameter monomials of degree at most B_1, and the constant-determinant cost inside L_1 is absorbed into the matrix-evaluation term. Substituting (12) into the two preferred determinant models yields

$$T_1^{\text{minor}} = \widetilde{O}(n 2^n \mu_1 S_1) = \widetilde{O}\left(n\, 2^n \binom{n-1+k+d}{n-1+k} \left(C_n^{(d)}\right)^2 \binom{nd+k}{k}\right), \tag{13}$$

$$T_1^{\text{sparse}} = \widetilde{O}(L_1 S_1 + S_1 \log q) = \widetilde{O}\left(n^2 \binom{n-1+k+d}{n-1+k} \left(C_n^{(d)}\right)^2 \binom{nd+k}{k}\right) \tag{14}$$

(the term $S_1 \log q$ is dominated by $L_1 S_1$ up to polylogarithmic factors). The sparse model saves a factor $2^n/n$ over expansion, but in all our experiments cached expansion by minors is faster, its constant factors being much smaller.

Step 2: Construct the Dixon matrix. Extracting the Dixon matrix coefficients from the cancellation determinant is dominated by Step 1. The block recursion of Zhao and Fu [82] builds $\mathcal{M}$ directly; following Qin et al. [66], the resulting surrogate T_2^{FDixon} is attractive when few variables are eliminated and degrees are high, but its $(n!)^3$ factor (Appendix H) makes it less competitive otherwise.

Step 3: Extract a maximal-rank submatrix. Step 3 extracts a maximal-rank submatrix numerically after finite-field evaluation, with cost

$$T_3 = \mathcal{O}\left(\left(C_n^{(d)}\right)^{\omega}\right). \tag{15}$$

The degree-aware weights of Section 4.1 are computed by a single pass over the $\left(C_n^{(d)}\right)^2$ entries, dominated by (15).

Step 4: Compute the Dixon resultant. This step computes the symbolic determinant of $\mathcal{M}'$, a polynomial matrix of size at most $C_n^{(d)}$ in the k retained parameters $\boldsymbol{a}$. In the Kronecker+HNF model, the complexity is governed by the entry-wise parameter degree $E_4 \le nd$, while in the evaluation-interpolation model it is governed by the total degree D_4 of the elimination polynomial. Under Heuristic 2, the elimination polynomial satisfies the Bézout degree bound $D_4 \le d^n$. The corresponding parameters are

$$N_4 \le (d^n+1)^k, \qquad L_4 = \widetilde{O}\Big(\big(C_n^{(d)}\big)^\omega\Big), \qquad \delta_4 = nd\,(C_n^{(d)}\cdot nd+1)^{k-1}, \quad (16)$$

where δ_4 denotes the average column degree after Kronecker substitution and reduces to $\delta_4 = nd$ when $k=1$. Therefore,

$$T_4^{\mathrm{HNF}} = \widetilde{O}\Big(\big(C_n^{(d)}\big)^\omega \delta_4\Big) = \widetilde{O}\Big(\big(C_n^{(d)}\big)^\omega nd(C_n^{(d)}\cdot nd+1)^{k-1}\Big), \quad (17)$$

$$T_4^{\mathrm{eval}} = \widetilde{O}(N_4(L_4+kD_4)) = \widetilde{O}\Big((d^n+1)^k\left(\big(C_n^{(d)}\big)^\omega + k\,d^n\right)\Big). \quad (18)$$

Overall comparison. Let T_1^{best} and T_4^{best} be the minimum cost across all considered models for Steps 1 and 4, respectively. Then $\mathcal{T}_{\mathrm{Dixon}} = \widetilde{O}(\max\{T_1^{\mathrm{best}}, T_4^{\mathrm{best}}\})$. At the idealized boundary $\omega = 2$, both stages must be retained. For $\omega > 2$, choosing the best applicable model at each stage makes Step 4 asymptotically dominant in the fixed-d or fixed-n regimes analyzed in Appendix H.3. It also dominates the running time in our experiments, so we use T_4^{best} as the asymptotic estimate.

Well-determined case. For $k=1$, the final determinant is a $C_n^{(d)} \times C_n^{(d)}$ univariate polynomial matrix for which the HNF algorithm of [58] is typically optimal:

$$T_4^{\mathrm{HNF}} = \widetilde{O}\Big(\big(C_n^{(d)}\big)^\omega nd\Big) = \widetilde{O}\left(nd\left(\frac{1}{n(d-1)+1}\binom{nd}{n}\right)^\omega\right). \quad (19)$$

For Step 1, the most competitive models are sparse interpolation and cached expansion by minors; substituting $k=1$ into (13) and (14) gives

$$T_1^{\mathrm{minor}} = \widetilde{O}\left(n^2\,d\,2^n\binom{n+d}{n}\big(C_n^{(d)}\big)^2\right), \quad (20)$$

$$T_1^{\mathrm{sparse}} = \widetilde{O}\left(n^3\,d\binom{n+d}{n}\big(C_n^{(d)}\big)^2\right). \quad (21)$$

With the best applicable construction model, Step 4 is asymptotically dominant for $\omega > 2$ in the regimes analyzed in Appendix H.3, and is also the empirical bottleneck. We therefore adopt $T = T_4^{\mathrm{HNF}}$ as the asymptotic estimate for well-determined systems; Appendix H.4 gives the model-specific comparisons.

Remark 6. For general $(d_1, \ldots, d_n)$-systems with $k = 1$, the same argument yields $\mathcal{T}_{\text{Dixon}} = \widetilde{O}\big(\big(\sum_{i=1}^{n} d_i\big)\, D(\mathcal{F})^{\omega}\big)$, with $D(\mathcal{F})$ given by Corollary 1(3).

3.3 Comparison with Gröbner Basis Methods

Gröbner basis methods solve polynomial systems by computing a grevlex order basis via F4/F5 [10,34] and then converting to lexicographic order via FGLM [35]. We compare Dixon with these methods on $(n; d)$-systems.

Table 3. Arithmetic cost bounds for well-determined $(n; d)$-systems, with $d_{\text{reg}} = n(d-1)+1$, $d_{\mathcal{I}} = d^n$, and $t = \Theta(d^{n-1}/\sqrt{n})$. Ratios omit logarithmic factors and use n fixed, $d \to \infty$; Stirling bounds simplify the factorial coefficients to display $e^{-n\omega}$. These are order estimates, not uniform joint asymptotic equivalents (Appendix D).

Method	Complexity	Ratio to Dixon
Dixon Resultant	$\widetilde{\mathcal{O}}\left(nd \cdot \left(\frac{1}{n(d-1)+1}\binom{nd}{n}\right)^{\omega}\right)$	1
GB (classical) [10]	$\mathcal{O}\left(\binom{n+d_{\text{reg}}}{d_{\text{reg}}}^{\omega}\right)$	$\mathcal{O}\left((nd)^{\omega-1}\right)$
GB (F5) [11,15]	$\mathcal{O}\left(n \cdot d_{\text{reg}} \cdot \binom{n+d_{\text{reg}}-1}{d_{\text{reg}}}^{\omega}\right)$	$\mathcal{O}\left(n^{\omega+1}\right)$
GB (F5 refined) [11]	$\mathcal{O}\left(n \cdot \binom{n+d_{\text{reg}}-d}{n} \cdot \binom{n+d_{\text{reg}}}{n}^{\omega-1}\right)$	$\mathcal{O}\left(n^{\omega} d^{\omega-1}\right)$
GB (per-deg) [71,57]	$\mathcal{O}\left(n \cdot \sum_{i=0}^{d_{\text{reg}}} \binom{n+i-d-1}{i-d}\binom{n+i-1}{i}^{\omega-1}\right)$	—
FGLM [35,57]	$\mathcal{O}\left(n \cdot d_{\mathcal{I}}^{\omega}\right)$	$\mathcal{O}\left(n^{3\omega/2} d^{\omega-1} e^{-n\omega}\right)$
BNS order change [17]	$\widetilde{\mathcal{O}}\left(t^{\omega-1} d_{\mathcal{I}}\right)$	$\mathcal{O}\left(n^{\omega-1/2} e^{-n\omega}\right)$

Well-Determined Systems ($m = n$) Numerous studies [10,11,15,71,57] have established theoretical complexity bounds for Gröbner basis algorithms. Table 3 summarizes these bounds and their ratios relative to Dixon.

The Gröbner-based approach includes both the grevlex basis computation and the change to lexicographic order, so the relevant comparison is with the combined F4/F5+FGLM cost. In the high-degree regime of the table, the displayed Dixon bound is asymptotically smaller than this combined bound. We also include the HNF-based change-of-order algorithm of [17] as an alternative to FGLM.

These Gröbner basis costs are upper bounds and do not fully capture practical F4/F5 optimizations. The ratios therefore compare theoretical estimates,

while Section 4.2 provides practical comparisons. Figure 1 evaluates the unsimplified cost expressions as n varies, including the per-degree sum; further plots and ratio derivations are given in Appendices E and D.

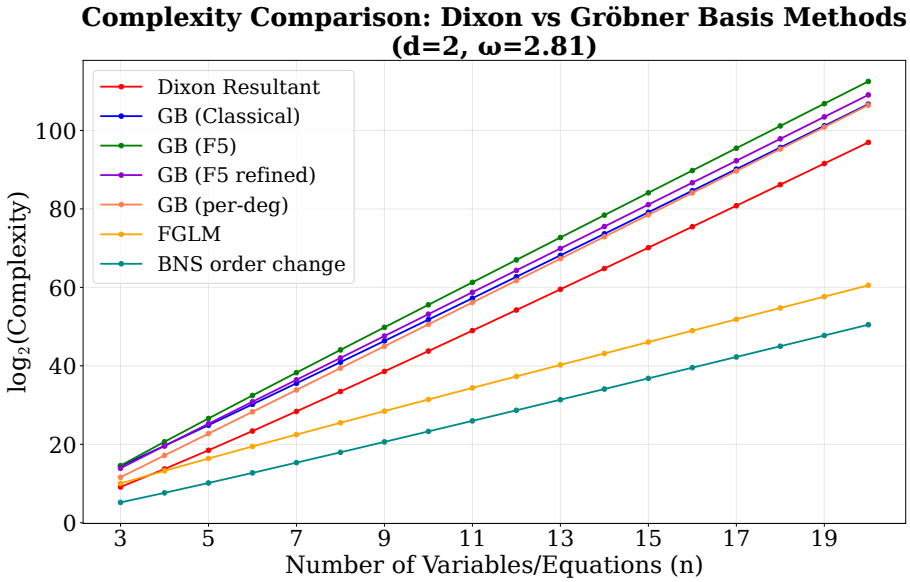


Fig. 1. Complexity comparison varying n with $d = 2$, $\omega = 2.81$.

Overdetermined Systems ($m > n$) Gröbner basis methods are typically preferable here: the degree of regularity is significantly lower than for well-determined systems [10], and the grevlex basis often yields univariate polynomials directly, eliminating the FGLM step. The Dixon resultant, in contrast, needs exactly n equations to eliminate $n-1$ variables and cannot leverage redundancy: one selects n equations and verifies the rest afterward, at slightly higher cost than the well-determined case.

Underdetermined Systems ($m < n$) For complete solving, the common cryptanalytic strategy is to guess $n-m$ variables and reduce to a (well- or over-)determined subsystem solved by Gröbner bases, and we do not propose Dixon as a replacement. From the viewpoint of elimination, however, Dixon removes several variables at once and produces explicit relations in fewer variables, which can then be combined with guessing or meet-in-the-middle decompositions; by contrast, changing term orders for positive-dimensional ideals typically requires Gröbner walks [28], whose complexity is hard to predict.

In summary, the two methods are complementary: Dixon and Gröbner basis are comparable on well-determined systems; Gröbner basis is typically preferable for overdetermined systems; Dixon is attractive for underdetermined systems as a determinant-based elimination method.

4 Optimized Implementation of the Dixon Resultant

While the theoretical complexity analysis highlights the advantages of Dixon resultant methods, bridging the gap to practical efficiency requires overcoming several significant hurdles. Currently, off-the-shelf computer algebra systems offer limited support: Dixon resultants are not natively implemented in advanced solvers such as Magma [22], and although implementations exist in Fermat [60] and through third-party packages in Maple [51,50], their performance is often unsatisfactory at scale. Beyond software limitations, extraneous factors must be carefully controlled or the classical Bézout bound becomes inapplicable, and large-scale polynomial-matrix operations must be handled efficiently. This section addresses these issues.

4.1 Eliminating Extraneous Factors via Degree-Aware Submatrix Selection

The primary challenge in implementing the Dixon resultant arises from the *extraneous factors* in Definition 2, which do not correspond to actual solutions of the original system and can substantially inflate the resultant degree beyond the Bézout bound (Theorem 1). Without extraneous factors, the degree is controlled by the ideal degree and hence by Bézout, providing both theoretical elegance and computational efficiency.

Chtcherba and Kapur reduce extraneous degree through support translations and dialytic multiplier selection [27, Section 9]. Qin et al. [67] proposed a method based on parameter specialization and regular-chain computation; however, the regular-chain step incurs additional overhead. Our strategy selects low-degree maximal-rank submatrices of the original Dixon matrix. Our experimental results show that the proposed greedy selection algorithm reduces redundant factors, and in most cases the resulting Dixon determinants remain at or below the Bézout bound.

Degree-aware submatrix selection strategy. A naive constant-evaluation rank test ignores the degree structure of the resulting submatrix. We develop a degree-aware selection strategy that employs a greedy approach to prioritize rows and columns with lower degrees. Following the determinant degree estimation framework of Labahn et al. [58], for any nonsingular polynomial matrix A,

$$\deg(\det(A)) \leq \min(|\mathrm{rdeg}(A)|, |\mathrm{cdeg}(A)|),$$

where $|\mathrm{rdeg}(A)|$ and $|\mathrm{cdeg}(A)|$ denote the sum of row degrees and column degrees, respectively. Our method is based on the following working hypothesis:

Heuristic 1 (Degree Minimization Heuristic) *For two polynomial matrices A and B of the same dimension over $\mathbb{K}[t_1, \ldots, t_m]$, if $\min(|rdeg(A)|, |cdeg(A)|) \geq \min(|rdeg(B)|, |cdeg(B)|)$, then we expect $\deg(\det(A)) \geq \deg(\det(B))$ to hold generically.*

The intuition is that a smaller row/column-sum degree bound tends to indicate a smaller determinant degree, motivating our greedy strategy to favor low-degree maximal-rank submatrices and thereby potentially reduce extraneous factors.

Remark 7. Labahn et al. [58] also used a refined bound $\deg(\det(A)) \leq \max_{\pi \in S_n} \sum_i \deg(a_{i,\pi(i)})$, where S_n is the symmetric group on $\{1, \ldots, n\}$. For a given matrix, this bound can be computed in $\mathcal{O}(n^3)$ by maximum-weight bipartite matching. We use the cheaper row/column weights as a heuristic for selecting low-degree submatrices.

Degree-aware selection algorithm. Based on Heuristic 1, we design a greedy algorithm that combines rank verification with degree-aware ordering of rows and columns. The core idea is to assign degree weights to rows and columns of the Dixon matrix and prioritize those with lower degrees during Gaussian elimination, ensuring the selected submatrix has maximal rank while heuristically favoring a low determinant degree. The full pseudocode is in Appendix F.

The rank test is performed at a random evaluation point, so the identified rank equals the true symbolic rank only with high probability; by the Schwartz–Zippel lemma [70,83], the failure probability is negligible, and a complete analysis in the same setting can be found in [52,50].

Heuristic 2 *Let $\mathcal{P} = \{p_1, \ldots, p_n\}$ be a generic polynomial system with degrees $d_1, \ldots, d_n$, and $\mathcal{M}$ its Dixon matrix. The submatrix selected by Algorithm 1 yields a resultant R satisfying $\deg(R) \leq \prod_{i=1}^{n} d_i$.*

This is supported by computational experiments (Appendix G), with rare observed violations of the Bézout bound by only 1–2 degrees. Passing this test does not establish the absence of extraneous factors or excess multiplicities. The estimates using $D_4 \leq \prod_i d_i$ assume Heuristic 2; the stronger bound $D_4 \leq d_{\mathcal{I}}$ additionally assumes their absence.

4.2 Implementation

We developed a comprehensive, open-source C implementation of the Dixon resultant method, `DRSolve`, built upon the high-performance `FLINT` [77] and `PML` [78,49] libraries, available at https://github.com/drsolve/drsolve (with AI coding assistance). It supports operations over prime fields $\mathbb{F}_p$ of arbitrary size, general finite fields $\mathbb{F}_q$, and $\mathbb{Q}$. It incorporates most Dixon construction techniques from [59,82,50] and most polynomial-matrix determinant techniques from [58,48,47,61]. It provides:

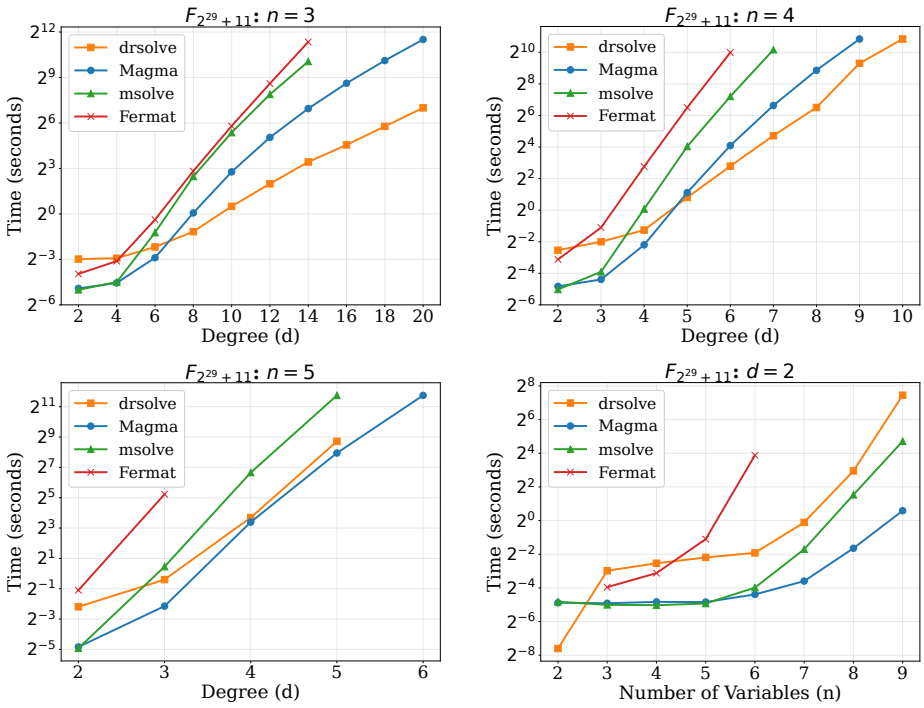


Fig. 2. Performance comparison (time in seconds) of `DRSolve` 0.3.0, Magma 2.29-7, msolve 0.9.5, and Fermat 7.9b on randomly generated dense polynomial systems over $\mathbb{F}_p$ with $p = 2^{29} + 11$. The four panels show degree sweeps for $n = 3, 4, 5$ and a variable-count sweep for $d = 2$; here n denotes the number of variables (equal to the number of equations). A curve stops where the corresponding solver exceeded one hour.

- **Elimination and solving:** Eliminate $n - 1$ variables from n polynomials, and compute all solutions of zero-dimensional systems.
- **Flexible determinant algorithms:** Support for all determinant computation methods discussed in this paper.
- **Optimized arithmetic:** Binary extension fields $\mathbb{F}_{2^8}, \mathbb{F}_{2^{16}}, \mathbb{F}_{2^{32}}, \mathbb{F}_{2^{64}}, \mathbb{F}_{2^{128}}$, and multi-modular techniques with CRT-based reconstruction over $\mathbb{Q}$ and large prime fields.
- **Complexity estimation:** A tool for estimating the cost of a polynomial system, or of a parameter set such as the number of variables and degrees.

Experimental results. Dixon resultants are well suited to parametric elimination, while Gröbner bases are particularly effective for overdetermined solving. We therefore focus our benchmarks on well-determined systems, retaining one variable as a parameter in Dixon elimination.

All benchmarks were performed on a dual-socket AMD EPYC 7302 server. We compared our Dixon implementation with the Gröbner basis solvers in

Magma [22]/msolve [16], and with the high-performance Dixon implementation Fermat [60]. Jinadu–Monagan's Maple implementation is designed for $\mathbb{Q}$ and is not included in our finite-field benchmarks. As shown in Figure 2, the results reveal a clear trade-off: Magma performs best for systems with more variables and lower degrees, possibly benefiting from F4/F5 optimizations, while `DRSolve` performs best for systems with fewer variables and higher degrees. Our advantage over Fermat is primarily due to the efficient polynomial-matrix routines in FLINT/PML, in particular the HNF algorithm.

5 Application to AO Primitives

This section organizes the applications around three elimination patterns. The goal is to highlight the solving methodology first and to use concrete AO primitives as representative case studies. We focus on solving strategies rather than on new algebraic models for AO primitives, reusing the best-known models from the literature and reevaluating them through the Dixon-resultant viewpoint.

Remark 8. Although much of the preceding discussion is developed under genericity assumptions, the structural bounds remain applicable to non-generic polynomial systems arising in AO primitives. Estimates relying on heuristic assumptions remain conditional. These bounds may be loose for structured instances, but still provide useful complexity estimates for algebraic analysis.

5.1 Method 1: Direct elimination

Method 1 applies the Dixon resultant directly to a square polynomial system. Starting from n equations in n variables, we eliminate $n-1$ variables, solve the resulting elimination polynomial in the remaining parameter, and then back-substitute recursively. The whole algebraic burden is concentrated in one structured determinant, so the complexity analysis reduces to the matrix-size and determinant models from Section 3.2. This is the classical Dixon solving workflow of [76,3], instantiated here with the bounds of Section 3 and the implementation of Section 4.

This method is best suited to Type I (low-degree) primitives, whose CICO systems admit a dense but uniformly low-degree algebraic description, so that the first elimination dominates and the later back-substitution phases operate on lower-dimensional systems; we take Poseidon as the representative example.

Application to Poseidon. We consider the CICO-3 problems of Poseidon [41] and Poseidon2 [42] in sponge mode, using the input/output and internal-subspace models from [43]; the precise models are summarized in Appendix J (see Figure 9 for an overview of the permutation). For the pure I/O model, direct Dixon elimination matches the natural square-system viewpoint. For the mixed-degree subspace models, the Hessenberg bound from Section 3 becomes essential because the polynomial degrees are no longer uniform. Figure 3 compares the

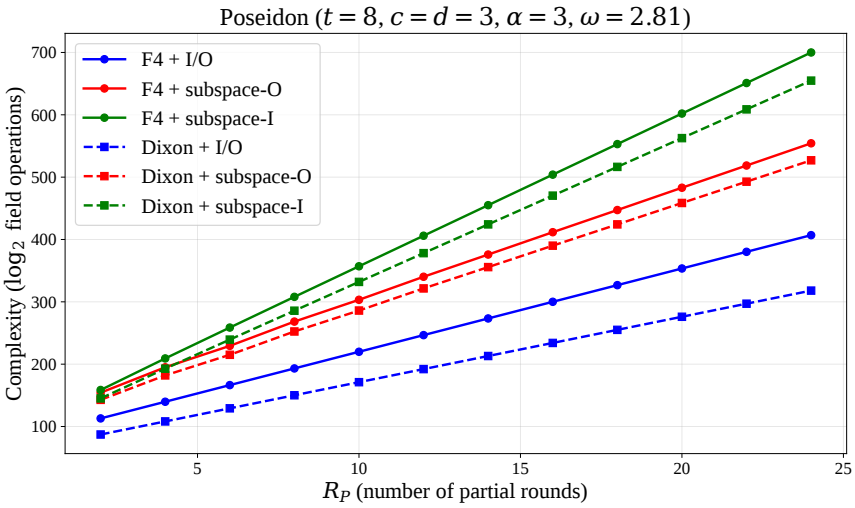


Fig. 3. Complexity comparison for Poseidon and Poseidon2, with the model parameters $t = 8$, $c = d = 3$, $\alpha = 3$ of [43]. Here $\omega = 2.81$ for all curves.

resulting complexities for $t = 8$, $c = d = 3$, $\alpha = 3$ across different numbers of partial rounds.

Table 4 reports small-instance timings. Full-round systems behave close to generic dense systems, while partial-round systems are much more structured, so both the Dixon matrix size and the Gröbner-basis degree of regularity fall well below their generic upper bounds. Since the size of this drop depends on the specific subspace model, the generic bound should be read as an upper-bound template rather than as a tight prediction of the best attack. A finer analysis of these structural effects is left for future work.

5.2 Method 2: Iterative elimination

Throughout this section calligraphic letters denote blocks: $\mathcal{F}, \mathcal{P}, \mathcal{R}$ are blocks of polynomials, $\mathcal{X}, \mathcal{Z}$ are blocks of variables, and $|\mathcal{X}|$ is the number of variables in the block $\mathcal{X}$.

Method 2 is designed for ladder-structured systems and generalizes iterative two-polynomial elimination to local multipolynomial subsystems. It can be viewed as a generalization of the resultant-based approach of Yang et al. [81] to multipolynomial Dixon elimination, and as an AO-oriented specialization of hybrid Dixon-matrix construction from [31]. We write such a system as

$$\mathcal{S}: \quad \mathcal{F}_1(\mathcal{X}_0, \mathcal{X}_1),\ \mathcal{F}_2(\mathcal{X}_1, \mathcal{X}_2),\ \ldots,\ \mathcal{F}_n(\mathcal{X}_{n-1}, \mathcal{X}_n), \tag{22}$$

with $|\mathcal{F}_1| = |\mathcal{X}_0| + 1$ and $|\mathcal{F}_i| = |\mathcal{X}_{i-1}|$ for $2 \le i \le n$. Eliminating successively,

$$R_1 = \mathrm{Res}_{\mathcal{X}_0}(\mathcal{F}_1),\ R_2 = \mathrm{Res}_{\mathcal{X}_1}(R_1, \mathcal{F}_2),\ \ldots,\ R_n = \mathrm{Res}_{\mathcal{X}_{n-1}}(R_{n-1}, \mathcal{F}_n), \tag{23}$$

Table 4. Timing comparison for POSEIDON instances ($t = 6, c = 1, d = 3$, I/O modeling, bypass one round) over $\mathbb{F}_p$, $p = 2^{62} + 169$. T and E denote theoretical and experimental values; complexities (Comp.) are in $\log_2$ field operations. Here $\omega = 2.81$. For odd R_F, one additional full round is added at the beginning. Both the Comp. and the timed implementation use the HNF determinant model.

Configuration	**Gröbner Basis**					**Dixon Resultant**				
	d_{reg}		Comp.		Time	$D(\mathcal{F})$		Comp.		Time
	T	E	T	E	(s)	T	E	T	E	(s)
$R_F = 3, R_P = 0, \alpha = 3$	25	25	32.8	32.8	2.2	117	108	24.1	23.7	0.3
$R_F = 3, R_P = 0, \alpha = 5$	73	71	45.2	44.9	94295	925	875	33.9	33.7	1477
$R_F = 4, R_P = 0, \alpha = 3$	79	79	46.2	46.2	188189	1080	1053	34.7	34.6	1772
$R_F = 2, R_P = 1, \alpha = 3$	25	11	32.8	23.9	0.02	117	36	24.1	19.3	0.006
$R_F = 2, R_P = 1, \alpha = 5$	73	27	45.2	33.7	14.3	925	175	33.9	27.2	2.1
$R_F = 3, R_P = 1, \alpha = 3$	79	31	46.2	35.3	178	1080	891	34.7	33.9	695
$R_F = 2, R_P = 2, \alpha = 3$	79	24	46.2	32.4	0.4	1080	297	34.7	29.4	11.9

yields a univariate eliminant if $|\mathcal{X}_n| = 1$, or a smaller system otherwise.

Assume each block $\mathcal{F}_j$ has common degree d_j. By Bézout, the ideal-theoretic eliminant produced at step i satisfies

$$\deg(R_i) \le \prod_{j=1}^{i} d_j^{|\mathcal{F}_j|}. \tag{24}$$

The last step produces R_n from R_{n-1} and $\mathcal{F}_n$, by eliminating the variables in $\mathcal{X}_{n-1}$. Here R_{n-1} has degree $D_{\text{in}} := \prod_{j=1}^{n-1} d_j^{|\mathcal{F}_j|}$ and each polynomial in $\mathcal{F}_n$ has degree d_n. Viewing this as Dixon elimination on one polynomial of degree D_{in} and $|\mathcal{F}_n|$ polynomials of degree d_n, the Unrestricted Bound (6) gives matrix size at most $\binom{d_n|\mathcal{F}_n| - d_n + D_{\text{in}}}{|\mathcal{F}_n|}$. The total $\mathcal{X}_n$-degree of the cancellation determinant is at most $D_{\text{in}} + d_n|\mathcal{F}_n|$ (sum of the per-row degrees in the eliminated block), which in turn upper-bounds the entry-wise $\mathcal{X}_n$-degree of the resulting Step 4 polynomial matrix (Section 3.2 Step 4), so HNF gives

$$\widetilde{\mathcal{O}}\left((D_{\text{in}} + d_n|\mathcal{F}_n|) \binom{d_n|\mathcal{F}_n| - d_n + D_{\text{in}}}{|\mathcal{F}_n|}^{\omega} \right), \tag{25}$$

with the Hessenberg Bound (8) available when the Unrestricted Bound is too coarse.

For meet-in-the-middle (MITM) instances, we split the chain into forward and backward parts meeting in a small middle block $\mathcal{Z}$:

$$\mathcal{F}_1(\mathcal{X}_0, \mathcal{X}_1), \ldots, \mathcal{F}_n(\mathcal{X}_{n-1}, \mathcal{Z}), \quad \mathcal{F}'_1(\mathcal{Y}_0, \mathcal{Y}_1), \ldots, \mathcal{F}'_m(\mathcal{Y}_{m-1}, \mathcal{Z}). \tag{26}$$

Forward and backward eliminations give two middle-state relations of degrees

$$D_{\mathrm{fwd}} = \prod_{j=1}^{n} d_j^{|\mathcal{F}_j|}, \qquad D_{\mathrm{bwd}} = \prod_{j=1}^{m} d_j'^{|\mathcal{F}_j'|}.$$

The final merge eliminates the middle block $\mathcal{Z}$ and, with $d_{\max} = \max(D_{\mathrm{fwd}}, D_{\mathrm{bwd}})$, admits the coarse bound

$$\widetilde{\mathcal{O}}\big(d_{\max}^{\omega+1}\big)\,. \tag{27}$$

The complexity of Method 2 is the maximum over all stages: every determinant of the forward and backward chains (25) and the final merge (27). Root finding, back-substitution, and the verification of the candidate solutions in the original system—which also discards the spurious candidates possibly introduced by extraneous factors—are negligible in comparison.

This method is best suited to Type II (equivalence-relation) primitives, whose ladder-structured CICO systems link successive states through local low-degree relations; we take `Vision` as the representative example.

Application to Vision. For `Vision` [4] with $s = 2$, inverse layers make the full direct model unattractive, so we adopt a meet-in-the-middle (MITM) approach following the general framework of Liu et al. [62] and apply iterative Dixon elimination. The detailed model and equation distribution are in Appendix K (the round function is recalled in Figure 10); Figure 4 and Table 5 summarize the outcome.

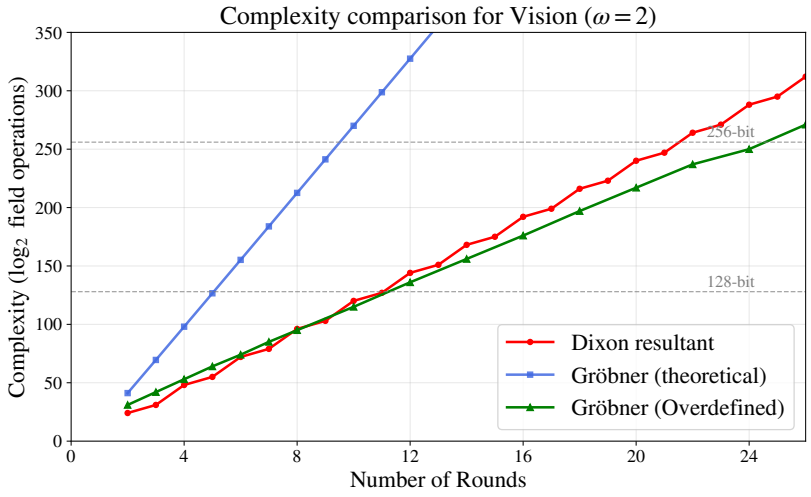


Fig. 4. Complexity comparison for `Vision` ($\omega = 2$ for consistency with [62]).

For these parameters the final merge (27) dominates every forward and backward stage of the chain, so it alone determines the reported complexity; the stage-by-stage figures, together with a rank bound that keeps the intermediate rungs below the merge, are collected in Appendix K. This is the structured regime where local elimination is preferable to direct elimination.

Table 5. Experimental timing and complexity comparison for `Vision` ($s = 2$). Complexities (Comp.) are in $\log_2$ field operations. $\omega = 2$ for consistency with [62].

	2 rounds		**3 rounds**		**4 rounds**	
Method	Comp.	Time	Comp.	Time	Comp.	Time
Gröbner basis [62]	31.0	1.38s	42.0	1d 4h	53.0	—
Dixon resultant	24.1	0.1s	31.0	15m	48.1	3d

5.3 Method 3: Ideal reduction and hybrid elimination

Method 3 combines Dixon elimination with the reduction machinery developed for FreeLunch-style systems in [13]. We start from a triangular system

$$\mathcal{P} = \begin{cases} z_1^{\alpha} - f_1(x) = 0, \\ z_2^{\alpha} - f_2(x, z_1) = 0, \\ \vdots \\ z_n^{\alpha} - f_n(x, z_1, \ldots, z_{n-1}) = 0, \\ h(x, z_1, \ldots, z_n) = 0, \end{cases} \tag{28}$$

and use the reduction procedure `Reduce` of [13, Algorithm 1]. For a polynomial g of weighted degree $d_{\mathcal{P}_k}(g)$ in a single retained variable, the reduction cost is $\widetilde{O}(d_{\mathcal{P}_k}(g)(2\alpha - 1)^n)$ [13, Lemma 1], where n is the number of levels in the triangular ideal $\mathcal{P}_k$. We take the reduced triangular system as input. If it must be constructed from an arithmetic circuit, its construction cost must be added separately [13, Proposition 4]. For CICO-1 and a system that is a FreeLunch Gröbner basis, the optimized two-polynomial strategy of [13, Corollary 3] applies directly and gives

$$\widetilde{O}\left(d_{\mathcal{I}}\alpha^2 \left(\frac{2\alpha - 1}{\alpha}\right)^{n+2}\right). \tag{29}$$

For general CICO-k, we retain $|\mathcal{X}| = k$ input variables and build k relations $R_1(\mathcal{X}), \ldots, R_k(\mathcal{X})$ in three phases.

(P1) Reduction. Each output constraint h_j is replaced by its normal form modulo the triangular ideal, using [13, Algorithm 1]. Since the polynomials manipulated here are k-variate in $\mathcal{X}$, the coefficient-count factor of [13, Lemma 1] is

replaced by the dense coefficient count $B_k(e) := \binom{e+k}{k}$ of a k-variate polynomial of degree e, which is $\Theta(e^k)$ for fixed k. Reduction never increases the weighted degree, so with $\bar{D} := \max_j d_{\mathcal{P}_k}(h_j)$ this phase costs

$$T_{\text{red}} = \widetilde{O}\big(k\, B_k(\bar{D})\, (2\alpha-1)^n\big)\,. \tag{30}$$

(P2) Successive elimination. Reduction only lowers the exponents $\deg_{z_i}$ below α; it does not by itself produce an element of $\mathbb{K}[\mathcal{X}]$. For each constraint, set $h_{j,n} = \mathrm{NF}_{\mathcal{P}_n}(h_j)$ and compute, for $i = n, \ldots, 1$, $h_{j,i-1} = \mathrm{NF}_{\mathcal{P}_{i-1}}(\mathrm{Res}_{z_i}(h_{j,i}, z_i^\alpha - f_i))$. Here $\mathcal{P}_i$ contains the first i triangular equations and $R_j = h_{j,0} \in \mathbb{K}[\mathcal{X}]$. By [13, Lemma 2] the stage with i triangular levels remaining costs $\widetilde{O}(D_i(2\alpha-1)^{i+2})$ for a single retained variable, where $D_i \le \alpha^{n-i}\bar{D}$ bounds the weighted degree before eliminating z_i at that stage [13, Lemma 3]; here the factor D_i is again replaced by $B_k(D_i)$. Summing over all stages and all k constraints,

$$T_{\text{elim}} = \widetilde{O}\left(k \sum_{i=1}^{n} B_k(D_i)\,(2\alpha-1)^{i+2}\right). \tag{31}$$

For fixed α and $k = 1$, the geometric sum gives the bound of [13, Proposition 7], dominated by the first stage $i = n$. For fixed $k \ge 2$, since $B_k(D_i) = \Theta(D_i^k)$ and $\alpha^k > 2\alpha - 1$, the sum is instead dominated by the last stage $i = 1$. Note also that (30) is always dominated by the term $i = n$ of (31).

(P3) Final elimination. Let $d_{\mathcal{R}} := \max_j \deg(R_j) \le \alpha^n \bar{D}$. Assuming that $\langle R_1, \ldots, R_k\rangle$ is zero-dimensional and the Dixon projection conditions hold, the final step eliminates $k-1$ variables from these k relations by mixed-degree Dixon elimination:

$$T_{\text{final}} = \widetilde{O}\big(\max\{T_1^{\text{best}}(\mathcal{R}),\, T_3(\mathcal{R}),\, k\, d_{\mathcal{R}}\, D(\mathcal{R})^\omega\}\big)\,, \tag{32}$$

where T_1^{best} and T_3 are the construction and rank-extraction costs of Section 3.2, and $D(\mathcal{R})$ is bounded by Corollary 1(3), which specializes to $C_k^{(d_{\mathcal{R}})}$ when all the R_j have the same degree.

The total symbolic hybrid cost is, up to a constant factor, the maximum of (30)–(32); in particular the reduction and elimination phases are not absorbed into the final Dixon cost in general. By contrast, direct elimination on the unreduced system of $n+k$ equations gives

$$\widetilde{O}\left(\max\left\{T_1^{\text{best}}(\mathcal{P}_k),\, T_3(\mathcal{P}_k),\, \Big(\sum_i d_i\Big) D(\mathcal{P}_k)^\omega\right\}\right), \tag{33}$$

again with $D(\mathcal{P}_k)$ from Corollary 1(3), since the constraints h_j do not have the same degree as the triangular equations; the last term becomes $(n+k)\alpha(C_{n+k}^{(\alpha)})^\omega$ when all equations have degree α. The comparison between the two routes determines whether the reduction phase is worthwhile.

This method is an improved attack on Type II (equivalence-relation) primitives, refining Method 2 by interleaving Dixon elimination with ideal reduction; we take `XHash12` as the representative example.

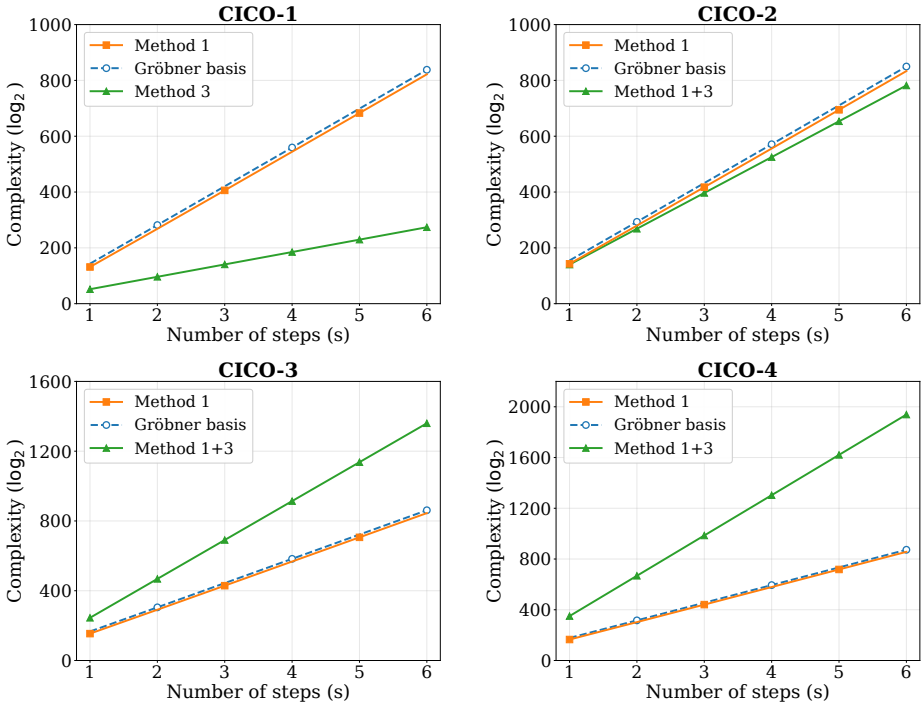


Fig. 5. Symbolic complexity comparison for `XHash12`, $\alpha = 7$ and $\omega = 2.81$, in $\log_2$ field operations. The CICO-1 Method 3 curve uses (29); the hybrid Method 1+3 for $k \geq 2$ includes reduction, successive elimination and final Dixon elimination. Each GB curve uses $\binom{m+d_{\mathrm{reg}}}{m}^{\omega}$, where $m = 12s + k$ and $d_{\mathrm{reg}} = m(\alpha - 1) + 1$.

Remark 9. For the special case of CICO-1, the two-polynomial iterative-resultant route of [13] already gives (29), and Dixon does not change the picture. In concurrent and independent work, Bak et al. [8] develop a related CICO-2 framework, introducing improvements to variable elimination and the final bivariate resultant, with substantial theoretical and practical gains. Although Method 3 does not currently offer a concrete advantage for $k \geq 3$, it provides a more flexible framework for further elimination strategies.

Application to `XHash12`. We focus on `XHash12` [5]; the permutation (Figure 11) and the triangular model we use are recalled in Appendix L. Figure 5 compares the direct Dixon method, the hybrid reduction-based strategy, and the corresponding generic Gröbner-basis estimate for each $k = 1, 2, 3, 4$. The hybrid method is excellent for CICO-1 and retains a small margin for CICO-2. For CICO-3 and CICO-4, however, the intermediate degrees make it worse than the direct elimination method.

Table 6 compares symbolic complexity estimates with recorded elimination times for small `XHash` polynomial systems. The special structure of these in-

stances and implementation details can lead to discrepancies between generic complexity bounds and measured running times. A refined complexity analysis accounting for this structure is left for future work.

Table 6. Elimination times and symbolic complexity estimates for small `XHash` polynomial systems. The tuples specify $(\alpha, \text{State}, \text{step})$: the exponent, state width, and number of steps. Comp. denotes $\log_2$ field operations with $\omega = 2.81$.

	CICO-1 $(3,3,5)$		CICO-1 $(7,3,5)$		CICO-2 $(3,4,3)$	
Method	Comp.	Time	Comp.	Time	Comp.	Time
Gröbner basis	48.8	1.5s	74.7	-	41.4	59.7s
Method 1	42.6	5.12s	64.4	-	35.1	4.5s
Method 1+3	18.6	0.04s	29.6	200.4s	31.2	413.2s

6 Conclusion

In this work, we revisited the Dixon resultant as a practical and theoretically grounded tool for algebraic cryptanalysis. By establishing upper bounds on the Dixon matrix size that are attained by the monomial support of generic systems, we derived refined complexity estimates. Combined with an efficient implementation and a degree-aware submatrix selection strategy, our framework enables reliable elimination for polynomial systems arising in cryptographic applications.

Our results show that Dixon resultants and Gröbner basis methods play complementary roles in algebraic cryptanalysis. In well-determined settings, Dixon resultants are comparable to Gröbner bases. Gröbner basis methods are more effective for overdetermined systems, whereas Dixon resultants provide a structured determinant-based elimination subroutine for underdetermined settings.

There remain several directions for future work.

1. Further analyzing AO primitives through better algebraic models for Dixon elimination, refined complexity analyses accounting for AO-specific structures, and applications to Type III (lookup-table) primitives in combination with recent algebraic techniques [9];
2. Precisely estimating the rank of the Dixon matrix to evaluate the Step 4 complexity exactly and optimize Step 3; improving Dixon-resultant techniques, including nonheuristic algorithms for extraneous-factor elimination and possible connections with multivariate subresultants [30];
3. Optimizing Dixon resultants for MQ systems, with potential applications to schemes such as UOV [56]. Although asymptotically favorable under the compared naive Gröbner-basis bounds, direct Dixon elimination is generally slower than Magma in our MQ experiments and theoretically less efficient than guessing-based hybrid methods [20]. Improving its theoretical or practical performance on MQ systems remains an open direction.

Acknowledgements. We would like to thank our advisors, Meiqin Wang and Kai Hu, for their guidance and support. We are grateful to all the anonymous reviewers for their valuable comments and suggestions. We thank Biran Lu for an elegant argument on the monomial-counting problem that inspired the proof in Section 3.1, particularly Lemma 2. We also thank Robert H. Lewis, Michael Monagan, Ayoola Jinadu, and Guoqiang Deng for their helpful correspondence regarding the implementation of the Dixon resultant.

This work is supported by the National Cryptologic Science Fund of China (Grant No. 2025NCSF01013), the National Key R&D Program of China (Grant No. 2024YFA1013000), the National Natural Science Foundation of China (Grant No. U2336207), the Key R&D Program of Shandong Province, China (Grant No. 2026CXPT037), and the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (Grant No. JYB2025XDXM114).

Claude and ChatGPT were used to assist with the implementation of the DR-Solve software and the language polishing of this paper. All proofs, algorithms, and mathematical derivations were developed by the authors. All AI-assisted code was reviewed and tested by the authors.

References

1. Albrecht, M.R., Grassi, L., Perrin, L., Ramacher, S., Rechberger, C., Rotaru, D., Roy, A., Schofnegger, M.: Feistel structures for MPC, and more. In: ESORICS 2019. Lecture Notes in Computer Science, vol. 11736, pp. 151–171. Springer (2019). https://doi.org/10.1007/978-3-030-29962-0_8
2. Albrecht, M.R., Grassi, L., Rechberger, C., Roy, A., Tiessen, T.: MiMC: Efficient encryption and cryptographic hashing with minimal multiplicative complexity. In: ASIACRYPT 2016. Lecture Notes in Computer Science, vol. 10031, pp. 191–219 (2016). https://doi.org/10.1007/978-3-662-53887-6_7
3. Albrecht, M.: Algebraic attacks on the Courtois toy cipher. Cryptologia **32**(3), 220–276 (2008). https://doi.org/10.1080/01611190802058139, https://doi.org/10.1080/01611190802058139
4. Aly, A., Ashur, T., Ben-Sasson, E., Dhooghe, S., Szepieniec, A.: Design of symmetric-key primitives for advanced cryptographic protocols. IACR Trans. Symmetric Cryptol. **2020**(3), 1–45 (2020). https://doi.org/10.13154/tosc.v2020.i3.1-45
5. Ashur, T., Al Kindi, Mahzoun, M.: XHash8 and XHash12: Efficient STARK-friendly hash functions. IACR Cryptol. ePrint Arch. p. 1045 (2023), https://eprint.iacr.org/2023/1045
6. Aval, J.: Multivariate Fuss-Catalan numbers. Discret. Math. **308**(20), 4660–4669 (2008). https://doi.org/10.1016/J.DISC.2007.08.100
7. Bak, A., Bariant, A., Boeuf, A., Briaud, P., Øygarden, M., Phanse, A.: The algebraic CheapLunch: Extending FreeLunch attacks on arithmetization-oriented primitives beyond CICO-1. IACR Cryptol. ePrint Arch. p. 2040 (2025), https://eprint.iacr.org/2025/2040
8. Bak, A., Bariant, A., Hostettler, M., Neiger, V.: Resultants meet resultant: Improving CICO-1 and CICO-2 attacks on ZK-friendly permutations. IACR Cryptol. ePrint Arch. **2026**, 1281 (2026), https://eprint.iacr.org/2026/1281

9. Bak, A., Jazeron, G., Galissant, P., Perrin, L.: Attacking split-and-lookup-based primitives using probabilistic polynomial system solving applications to round-reduced Monolith and full-round Skyscraper. IACR Trans. Symmetric Cryptol. **2025**(3), 337–367 (2025). https://doi.org/10.46586/TOSC.V2025.I3.337-367, https://doi.org/10.46586/tosc.v2025.i3.337-367
10. Bardet, M., Faugère, J.C., Salvy, B.: On the complexity of Gröbner basis computation of semi-regular overdetermined algebraic equations. In: Proceedings of the International Conference on Polynomial System Solving (ICPSS). pp. 71–74 (2004), https://api.semanticscholar.org/CorpusID:7982329
11. Bardet, M., Faugère, J., Salvy, B.: On the complexity of the F5 Gröbner basis algorithm. J. Symb. Comput. **70**, 49–70 (2015). https://doi.org/10.1016/J.JSC.2014.09.025
12. Bareiss, E.H.: Sylvester's identity and multistep integer-preserving Gaussian elimination. Mathematics of Computation **22**(103), 565–578 (1968). https://doi.org/10.1090/S0025-5718-1968-0226829-0
13. Bariant, A., Boeuf, A., Briaud, P., Hostettler, M., Øygarden, M., Raddum, H.: Improved resultant attack against arithmetization-oriented primitives. In: Kalai, Y.T., Kamara, S.F. (eds.) Advances in Cryptology - CRYPTO 2025 - 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2025, Proceedings, Part V. Lecture Notes in Computer Science, vol. 16004, pp. 335–367. Springer (2025). https://doi.org/10.1007/978-3-032-01901-1_11
14. Bariant, A., Boeuf, A., Lemoine, A., Ayala, I.M., Øygarden, M., Perrin, L., Raddum, H.: The algebraic FreeLunch: Efficient Gröbner basis attacks against arithmetization-oriented primitives. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 14923, pp. 139–173. Springer (2024). https://doi.org/10.1007/978-3-031-68385-5_5
15. Bariant, A., Bouvier, C., Leurent, G., Perrin, L.: Algebraic attacks against some arithmetization-oriented primitives. IACR Trans. Symmetric Cryptol. **2022**(3), 73–101 (2022). https://doi.org/10.46586/TOSC.V2022.I3.73-101
16. Berthomieu, J., Eder, C., Safey El Din, M.: msolve: A library for solving polynomial systems. In: Proceedings of the 2021 on International Symposium on Symbolic and Algebraic Computation. pp. 51–58. ISSAC '21, ACM (2021). https://doi.org/10.1145/3452143.3465545
17. Berthomieu, J., Neiger, V., Safey El Din, M.: Faster change of order algorithm for Gröbner bases under shape and stability assumptions. In: Proceedings of the 2022 International Symposium on Symbolic and Algebraic Computation, ISSAC 2022. pp. 409–418. ACM (2022). https://doi.org/10.1145/3476446.3535484, https://hal.sorbonne-universite.fr/hal-03580736
18. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: Sponge functions. In: ECRYPT Hash Workshop (2007)
19. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: On the indifferentiability of the sponge construction. In: Smart, N.P. (ed.) Advances in Cryptology - EUROCRYPT 2008, 27th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Istanbul, Turkey, April 13-17, 2008. Proceedings. Lecture Notes in Computer Science, vol. 4965, pp. 181–197. Springer (2008). https://doi.org/10.1007/978-3-540-78967-3_11
20. Bettale, L., Faugère, J., Perret, L.: Hybrid approach for solving multivariate systems over finite fields. J. Math. Cryptol. **3**(3), 177–197 (2009). https://doi.org/10.1515/JMC.2009.009, https://doi.org/10.1515/JMC.2009.009

21. Bona, M.: Handbook of Enumerative Combinatorics. Discrete Mathematics and Its Applications, CRC Press (2015), https://books.google.com.tw/books?id=j3kZBwAAQBAJ
22. Bosma, W., Cannon, J., Playoust, C.: The Magma algebra system I: The user language. Journal of Symbolic Computation **24**(3-4), 235–265 (1997). https://doi.org/10.1006/jsco.1996.0125
23. Bouvier, C.: Cryptanalysis and design of symmetric primitives defined over large finite fields. (Cryptanalyse et conception de primitives symétriques définies sur de grands corps finis). Ph.D. thesis, Sorbonne University, Paris, France (2023), https://tel.archives-ouvertes.fr/tel-04327955
24. Bouvier, C., Briaud, P., Chaidos, P., Perrin, L., Salen, R., Velichkov, V., Willems, D.: New design techniques for efficient arithmetization-oriented hash functions: Anemoi permutations and Jive compression mode. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part III. Lecture Notes in Computer Science, vol. 14083, pp. 507–539. Springer (2023). https://doi.org/10.1007/978-3-031-38548-3_17
25. Cahill, N.D., D'Errico, J.R., Narayan, D.A., Narayan, J.Y.: Fibonacci determinants. The College Mathematics Journal **33**(3), 221–225 (2002)
26. Chtcherba, A.D., Kapur, D.: Constructing Sylvester-type resultant matrices using the Dixon formulation. Journal of Symbolic Computation **38**(1), 777–814 (2004). https://doi.org/10.1016/j.jsc.2003.11.003
27. Chtcherba, A.D., Kapur, D.: Resultants for unmixed bivariate polynomial systems produced using the Dixon formulation. Journal of Symbolic Computation **38**, 915–958 (2004). https://doi.org/10.1016/j.jsc.2003.12.001
28. Collart, S., Kalkbrener, M., Mall, D.: Converting bases with the Gröbner walk. J. Symb. Comput. **24**(3/4), 465–469 (1997). https://doi.org/10.1006/JSCO.1996.0145
29. Cox, D., Little, J., O'Shea, D.: Using Algebraic Geometry. Graduate Texts in Mathematics, Springer New York, 2 edn. (2005), https://books.google.com.tw/books?id=1blxizOS9N0C
30. Cox, D., D'Andrea, C.: Subresultants and the shape lemma. Math. Comput. **92**(343), 2355–2379 (2023). https://doi.org/10.1090/MCOM/3840
31. Deng, G., Qi, N., Tang, M., Duan, X.: Constructing Dixon matrix for sparse polynomial equations based on hybrid and heuristics scheme. Symmetry **14**(6), 1174 (2022). https://doi.org/10.3390/sym14061174
32. Dixon, A.L.: The eliminant of three quantics in two independent variables. Proceedings of the London Mathematical Society **s2-7**(1), 49–69 (1909). https://doi.org/10.1112/plms/s2-7.1.49, https://londmathsoc.onlinelibrary.wiley.com/doi/abs/10.1112/plms/s2-7.1.49
33. Faugère, J.C.: A new efficient algorithm for computing Gröbner bases (F4). Journal of Pure and Applied Algebra **139**(1-3), 61–88 (1999)
34. Faugère, J.C.: A new efficient algorithm for computing Gröbner bases without reduction to zero (F5). In: Proceedings of the 2002 International Symposium on Symbolic and Algebraic Computation. pp. 75–83. ISSAC '02, Association for Computing Machinery, New York, NY, USA (2002). https://doi.org/10.1145/780506.780516
35. Faugère, J., Gianni, P.M., Lazard, D., Mora, T.: Efficient computation of zero-dimensional Gröbner bases by change of ordering. J. Symb. Comput. **16**(4), 329–344 (1993). https://doi.org/10.1006/JSCO.1993.1051

36. Fulton, W.: Intersection Theory. Ergebnisse der Mathematik und ihrer Grenzgebiete, Springer, 2 edn. (1998). https://doi.org/10.1007/978-1-4612-1700-8
37. Gentleman, W.M., Johnson, S.C.: Analysis of algorithms, a case study: Determinants of matrices with polynomial entries. ACM Transactions on Mathematical Software **2**(3), 232–241 (1976)
38. Grassi, L., Hao, Y., Rechberger, C., Schofnegger, M., Walch, R., Wang, Q.: Horst meets fluid-SPN: Griffin for zero-knowledge applications. In: Handschuh, H., Lysyanskaya, A. (eds.) Advances in Cryptology - CRYPTO 2023. Lecture Notes in Computer Science, vol. 14083, pp. 573–606. Springer (2023). https://doi.org/10.1007/978-3-031-38548-3_19
39. Grassi, L., Khovratovich, D., Lüftenegger, R., Rechberger, C., Schofnegger, M., Walch, R.: Reinforced Concrete: A fast hash function for verifiable computation. In: Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022. pp. 1323–1335. ACM (2022). https://doi.org/10.1145/3548606.3560686
40. Grassi, L., Khovratovich, D., Lüftenegger, R., Rechberger, C., Schofnegger, M., Walch, R.: Monolith: Circuit-friendly hash functions with new nonlinear layers for fast and constant-time implementations. IACR Trans. Symmetric Cryptol. 2024(3); Cryptology ePrint Archive, Paper 2023/1025 (2024), https://eprint.iacr.org/2023/1025
41. Grassi, L., Khovratovich, D., Rechberger, C., Roy, A., Schofnegger, M.: Poseidon: A new hash function for zero-knowledge proof systems. In: Bailey, M.D., Greenstadt, R. (eds.) 30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021. pp. 519–535. USENIX Association (2021), https://www.usenix.org/conference/usenixsecurity21/presentation/grassi
42. Grassi, L., Khovratovich, D., Schofnegger, M.: Poseidon2: A faster version of the Poseidon hash function. In: Mrabet, N.E., Feo, L.D., Duquesne, S. (eds.) Progress in Cryptology - AFRICACRYPT 2023 - 14th International Conference on Cryptology in Africa, Sousse, Tunisia, July 19-21, 2023, Proceedings. Lecture Notes in Computer Science, vol. 14064, pp. 177–203. Springer (2023). https://doi.org/10.1007/978-3-031-37679-5_8
43. Grassi, L., Koschatko, K., Rechberger, C.: Poseidon and Neptune: Gröbner basis cryptanalysis exploiting subspace trails. IACR Trans. Symmetric Cryptol. **2025**(2), 34–86 (2025). https://doi.org/10.46586/TOSC.V2025.I2.34-86
44. Grassi, L., Onofri, S., Pedicini, M., Sozzi, L.: Invertible quadratic non-linear layers for MPC-/FHE-/ZK-friendly schemes over $\mathbb{F}_p^n$ application to Poseidon. IACR Trans. Symmetric Cryptol. **2022**(3), 20–72 (2022). https://doi.org/10.46586/TOSC.V2022.I3.20-72
45. Heintz, J.: Definability and fast quantifier elimination in algebraically closed fields. Theor. Comput. Sci. **24**, 239–277 (1983). https://doi.org/10.1016/0304-3975(83)90002-6
46. Hilton, P., Pedersen, J.: Catalan numbers, their generalization, and their uses. The Mathematical Intelligencer **13**(2), 64–75 (1991)
47. Horowitz, E., Sahni, S.: On computing the exact determinant of matrices with polynomial entries. Journal of the ACM **22**(1), 38–50 (1975). https://doi.org/10.1145/321864.321868
48. Huang, Q.: Sparse polynomial interpolation based on derivatives. J. Symb. Comput. **114**, 359–375 (2023). https://doi.org/10.1016/J.JSC.2022.06.002
49. Hyun, S.G., Neiger, V., Schost, É.: Implementations of efficient univariate polynomial matrix algorithms and application to bivariate resultants. In: Proceed-

ings ISSAC 2019. pp. 235–242. ACM (2019). https://doi.org/10.1145/3326229.3326272, https://github.com/vneiger/pml
50. Jinadu, A., Monagan, M.: A new Dixon resultant algorithm for solving parametric polynomial systems using sparse multivariate rational function interpolation. Preprint (2025), https://www.cecm.sfu.ca/CAG/papers/AyoolaJSC25.pdf
51. Jinadu, A., Monagan, M.B.: An interpolation algorithm for computing Dixon resultants. In: Boulier, F., England, M., Sadykov, T.M., Vorozhtsov, E.V. (eds.) Computer Algebra in Scientific Computing - 24th International Workshop, CASC 2022, Gebze, Turkey, August 22-26, 2022, Proceedings. Lecture Notes in Computer Science, vol. 13366, pp. 185–205. Springer (2022). https://doi.org/10.1007/978-3-031-14788-3_11
52. Jinadu, A.I.: Solving Parametric Systems Using Dixon Resultants and Sparse Interpolation Tools. Ph.D. thesis, Simon Fraser University (Summer 2023)
53. Kapur, D., Saxena, T.: Sparsity considerations in Dixon resultants. In: Miller, G.L. (ed.) Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing, Philadelphia, Pennsylvania, USA, May 22-24, 1996. pp. 184–191. ACM (1996). https://doi.org/10.1145/237814.237862
54. Kapur, D., Saxena, T.: Extraneous factors in the Dixon resultant formulation. In: Char, B.W., Wang, P.S., Küchlin, W. (eds.) Proceedings of the 1997 International Symposium on Symbolic and Algebraic Computation, ISSAC 1997, Maui, Hawaii, USA, July 21-23, 1997. pp. 141–148. ACM (1997). https://doi.org/10.1145/258726.258768
55. Kapur, D., Saxena, T., Yang, L.: Algebraic and geometric reasoning using Dixon resultants. In: MacCallum, M.A.H. (ed.) Proceedings of the International Symposium on Symbolic and Algebraic Computation, ISSAC '94, Oxford, UK, July 20-22, 1994. pp. 99–107. ACM (1994). https://doi.org/10.1145/190347.190372
56. Kipnis, A., Patarin, J., Goubin, L.: Unbalanced oil and vinegar signature schemes. In: Stern, J. (ed.) Advances in Cryptology - EUROCRYPT '99, International Conference on the Theory and Application of Cryptographic Techniques, Prague, Czech Republic, May 2-6, 1999, Proceeding. Lecture Notes in Computer Science, vol. 1592, pp. 206–222. Springer (1999). https://doi.org/10.1007/3-540-48910-X_15, https://doi.org/10.1007/3-540-48910-X_15
57. Koschatko, K., Lüftenegger, R., Rechberger, C.: Exploring the six worlds of Gröbner basis cryptanalysis: Application to Anemoi. IACR Trans. Symmetric Cryptol. **2024**(4), 138–190 (2024). https://doi.org/10.46586/TOSC.V2024.I4.138-190
58. Labahn, G., Neiger, V., Zhou, W.: Fast, deterministic computation of the Hermite normal form and determinant of a polynomial matrix. J. Complex. **42**, 44–71 (2017). https://doi.org/10.1016/J.JCO.2017.03.003
59. Lewis, R.H.: Dixon-EDF: The premier method for solution of parametric polynomial systems. In: Kotsireas, I.S., Martínez-Moro, E. (eds.) Applications of Computer Algebra, Springer Proceedings in Mathematics & Statistics, vol. 198, pp. 237–256. Springer International Publishing, Cham (2017). https://doi.org/10.1007/978-3-319-56932-1_16
60. Lewis, R.H.: Fermat computer algebra system. https://home.bway.net/lewis/ (2024), version 6.5
61. Li, Y.: An effective algorithm of computing symbolic determinants with multivariate polynomial entries. Applied Mathematics and Computation **192**(2), 382–388 (2007)

62. Liu, F., Mahzoun, M., Meier, W.: Modelling ciphers with overdefined systems of quadratic equations: Application to friday, vision, RAIN and biscuit. In: Chung, K., Sasaki, Y. (eds.) Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VII. Lecture Notes in Computer Science, vol. 15490, pp. 424–456. Springer (2024). https://doi.org/10.1007/978-981-96-0941-3_14
63. Macaulay, F.S.: Some formulae in elimination. Proceedings of the London Mathematical Society **1**(1), 3–27 (1902)
64. Meurant, G.: Hessenberg and Tridiagonal Matrices: Theory and Examples. SIAM (2025), https://epubs.siam.org/doi/10.1137/1.9781611978452
65. Painvin, L.: Note sur la méthode d'élimination de Bézout. Nouvelles annales de mathématiques: journal des candidats aux écoles polytechnique et normale **13**, 278–285 (1874)
66. Qin, X., Wu, D., Tang, L., Ji, Z.: Complexity of constructing Dixon resultant matrix. Int. J. Comput. Math. **94**(10), 2074–2088 (2017). https://doi.org/10.1080/00207160.2016.1276572
67. Qin, X., Zhang, L., Yang, L., Cao, S.: Heuristics to sift extraneous factors in Dixon resultants. J. Symb. Comput. **112**, 105–121 (2022). https://doi.org/10.1016/J.JSC.2022.01.003
68. Roy, A., Steiner, M.J., Trevisani, S.: Arion: Arithmetization-oriented permutation and hashing from generalized triangular dynamical systems. CoRR **abs/2303.04639** (2023). https://doi.org/10.48550/ARXIV.2303.04639
69. Sauer, J.F., Szepieniec, A.: SoK: Gröbner basis algorithms for arithmetization oriented ciphers. IACR Cryptol. ePrint Arch. p. 870 (2021), https://eprint.iacr.org/2021/870
70. Schwartz, J.T.: Fast probabilistic algorithms for verification of polynomial identities. Journal of the ACM (JACM) **27**(4), 701–717 (1980)
71. Spaenlehauer, P.: Solving multi-homogeneous and determinantal systems: algorithms, complexity, applications. (Résolution de systèmes multi-homogènes et déterminantiels : algorithmes, complexité, applications). Ph.D. thesis, Pierre and Marie Curie University, Paris, France (2012), https://tel.archives-ouvertes.fr/tel-01110756
72. Steiner, M.J.: Solving degree bounds for iterated polynomial systems. IACR Trans. Symmetric Cryptol. **2024**(1), 357–411 (2024). https://doi.org/10.46586/TOSC.V2024.I1.357-411
73. Storjohann, A.: Algorithms for matrix canonical forms. Ph.D. thesis, ETH Zurich (2000)
74. Sturmfels, B.: Solving Systems of Polynomial Equations. Conference Board of the Mathematical Sciences Regional Conference Series in Mathematics, Conference Board of the Mathematical Sciences (2002), https://books.google.com.tw/books?id=WUAjMQAACAAJ
75. Szepieniec, A., Lemmens, A., Sauer, J.F., Threadbare, B., Al-Kindi: The Tip5 hash function for recursive STARKs. Cryptology ePrint Archive, Paper 2023/107 (2023), https://eprint.iacr.org/2023/107
76. Tang, X., Feng, Y.: A new efficient algorithm for solving systems of multivariate polynomial equations. Cryptology ePrint Archive, Paper 2005/312 (2005), https://eprint.iacr.org/2005/312
77. The FLINT team: FLINT: Fast Library for Number Theory (2026), version 3.6.0, https://flintlib.org

78. The PML team: PML: Polynomial Matrix Library (2025), version 0.5, https://github.com/vneiger/pml
79. Villard, G.: On computing the resultant of generic bivariate polynomials. In: Proceedings of the 2018 International Symposium on Symbolic and Algebraic Computation, ISSAC 2018. pp. 391–398. ACM (2018). https://doi.org/10.1145/3208976.3209020, https://perso.ens-lyon.fr/gilles.villard/BIBLIOGRAPHIE/PDF/vil18.pdf
80. Wang, D.: Solving polynomial equations: Characteristic sets and triangular systems. Mathematics and Computers in Simulation **42**(4-6), 339–351 (1996). https://doi.org/10.1016/S0378-4754(96)00008-0
81. Yang, H., Zheng, Q., Yang, J., Liu, Q., Tang, D.: A new security evaluation method based on resultant for arithmetic-oriented algorithms. In: Chung, K., Sasaki, Y. (eds.) Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part VII. Lecture Notes in Computer Science, vol. 15490, pp. 457–489. Springer (2024). https://doi.org/10.1007/978-981-96-0941-3_15
82. Zhao, S., Fu, H.: An extended fast algorithm for constructing the Dixon resultant matrix. Science in China Series A: Mathematics **48**(1), 131–143 (2005)
83. Zippel, R.: Probabilistic algorithms for sparse polynomials. In: International symposium on symbolic and algebraic manipulation. pp. 216–226. Springer (1979)

Supporting Material

Table of Contents

A Background on Algebraic Geometry and Combinatorics

In this appendix, we provide essential background on convex geometry, polynomial support structures, and lattice path enumeration that facilitate understanding of the complexity analysis in Section 3. For a detailed introduction to these topics, please refer to [29,74,21].

Definition 5 (Generic Property [29]). *Fix degree bounds $d_1, \ldots, d_n$, and let $\mathcal{C}$ be the affine space over $\overline{\mathbb{K}}$ of the coefficient tuples of polynomial systems $(f_1, \ldots, f_n)$ with $\deg_{\boldsymbol{x}}(f_i) \leq d_i$. A property $\mathcal{P}$ is said to hold* generically *if there exists a Zariski open dense subset $U \subseteq \mathcal{C}$ such that $\mathcal{P}$ holds for every $(f_1, \ldots, f_n) \in U$.*

Equivalently, the property holds whenever a suitable nonzero polynomial in the input coefficients does not vanish; see [29, Chapter 3, §5, Definition (5.6), p. 115]. The exceptional systems are contained in a proper algebraic subset. Over a finite base field, this definition is interpreted over its algebraic closure and does not by itself assert a probability of success for sampling in that finite field.

Definition 6 (Convex Set and Convex Hull). *A set $C \subset \mathbb{R}^n$ is* convex *if for any two points $p, q \in C$, the line segment $\{(1-\lambda)p + \lambda q : \lambda \in [0,1]\}$ lies entirely in C.*

For any set $S \subset \mathbb{R}^n$, its convex hull *$conv(S)$ is the smallest convex set containing S, equivalently:*

$$conv(S) = \left\{ \sum_{i=1}^{m} \lambda_i s_i : s_i \in S, \lambda_i \geq 0, \sum_{i=1}^{m} \lambda_i = 1 \right\}.$$

Linear combinations of this form are called convex combinations.

Definition 7 (Polytope). *A* polytope *is the convex hull of a finite set in $\mathbb{R}^n$. A* lattice polytope *is a polytope whose vertices lie in $\mathbb{Z}^n$.*

Definition 8 (Monomial Support). *For a polynomial $f = \sum_\alpha c_\alpha x^\alpha$ where $c_\alpha \in \mathbb{K}$ and $\alpha \in \mathbb{Z}^n_{\geq 0}$, the* support *of f is the set of exponent vectors corresponding to nonzero coefficients:*

$$Supp(f) = \{\alpha \in \mathbb{Z}^n_{\geq 0} : c_\alpha \neq 0\}.$$

Definition 9 (Newton Polytope). *The* Newton polytope *of a polynomial f is the convex hull of its support:*

$$NP(f) = conv(Supp(f)) \subset \mathbb{R}^n.$$

The Newton polytope records the "shape" or sparsity structure of f; the actual coefficient values do not affect $NP(f)$.

Definition 10 (Mixed and Unmixed Systems [53]). *A polynomial system* $\mathcal{F} = (f_1, \ldots, f_n)$ *is called* unmixed *if*

$$\mathrm{NP}(f_1) = \cdots = \mathrm{NP}(f_n).$$

Otherwise, it is called mixed.

Definition 11 (Minkowski Sum). *For polytopes* $P_1, P_2 \subset \mathbb{R}^n$*, their Minkowski sum is:*

$$P_1 + P_2 = \{p_1 + p_2 : p_1 \in P_1, p_2 \in P_2\}.$$

More generally, for $\lambda \geq 0$ *and polytope* P*, define* $\lambda P = \{\lambda p : p \in P\}$*.*

Proposition 1 (Properties of Minkowski Sum).

1. *The Minkowski sum of polytopes is a polytope.*
2. *The Minkowski sum of lattice polytopes is a lattice polytope.*
3. *For polytopes* P, Q *and* $\lambda, \mu \geq 0$*:* $\lambda(P + Q) = \lambda P + \lambda Q$ *and* $(\lambda + \mu)P = \lambda P + \mu P$*.*

Proposition 2 (Newton Polytope of Products [53, Fact 3.2]). *For polynomials* $f, g \in \mathbb{K}[x_1, \ldots, x_n]$*:*

$$NP(fg) = NP(f) + NP(g).$$

Proposition 3 (Faces of Minkowski Sums). *Let* $P_1, \ldots, P_r \subset \mathbb{R}^n$ *be polytopes, and let* $P = P_1 + \cdots + P_r$ *be their Minkowski sum. Then every face* P' *of* P *can be expressed as a Minkowski sum:*

$$P' = P'_1 + \cdots + P'_r,$$

where each P'_i *is a face of* P_i*.*

Definition 12 (Lattice Path). *A* lattice path *in* $\mathbb{Z}^2_{\geq 0}$ *from* $(0, 0)$ *to* (u, w) *is a sequence of points*

$$(x_0, y_0), (x_1, y_1), \ldots, (x_k, y_k)$$

such that $(x_0, y_0) = (0, 0)$*,* $(x_k, y_k) = (u, w)$*, and for each* $i = 0, 1, \ldots, k - 1$ *we have either*

$$(x_{i+1}, y_{i+1}) = (x_i + 1, y_i) \quad \textit{(a right step)}$$

or

$$(x_{i+1}, y_{i+1}) = (x_i, y_i + 1) \quad \textit{(an up step).}$$

All points of the path lie in $\mathbb{Z}^2_{\geq 0}$*. Such a path consists of exactly* u *horizontal steps and* w *upward steps; the* height *of a horizontal step is the* y*-coordinate at which it is taken. The number of lattice paths refers to the total number of distinct lattice paths satisfying these conditions.*

B Proof of Lemma 2

Proof. We establish the equality by showing both inclusions.

Setup. For each $k \in \{1, \ldots, n\}$, define

$$P_k := \{(i_1, \ldots, i_k, 0, \ldots, 0) \in \mathbb{Z}^n_{\geq 0} : i_1 + \cdots + i_k \leq d_k\}.$$

By definition of the Iterated-Simplex Polynomial, whose coefficients over $\mathbb{Z}$ are positive,

$$\mathrm{Supp}(I_{d_1,d_2,\ldots,d_n}) = P_1 + P_2 + \cdots + P_n,$$

where $+$ denotes the Minkowski sum. Define the target set:

$$Q_n := \left\{ (\alpha_1, \ldots, \alpha_n) \in \mathbb{Z}^n_{\geq 0} : \sum_{\ell=n-j+1}^{n} \alpha_\ell \leq \sum_{i=1}^{j} d_{n-i+1} \text{ for all } j \in \{1, \ldots, n\} \right\}.$$

Inclusion $P_1 + \cdots + P_n \subseteq Q_n$. Let $(\alpha_1, \ldots, \alpha_n) \in P_1 + \cdots + P_n$. Then there exist $(i_1^{(k)}, \ldots, i_n^{(k)}) \in P_k$ for $k = 1, \ldots, n$ such that

$$(\alpha_1, \ldots, \alpha_n) = \sum_{k=1}^{n} (i_1^{(k)}, \ldots, i_n^{(k)}).$$

Fix $j \in \{1, \ldots, n\}$ and consider the last j components. For each $k \in \{n-j+1, \ldots, n\}$, by the definition of P_k, we have

$$\sum_{\ell=n-j+1}^{n} i_\ell^{(k)} \leq d_k.$$

Therefore,

$$\sum_{\ell=n-j+1}^{n} \alpha_\ell = \sum_{\ell=n-j+1}^{n} \sum_{k=1}^{n} i_\ell^{(k)} = \sum_{k=n-j+1}^{n} \sum_{\ell=n-j+1}^{n} i_\ell^{(k)}$$

$$\leq \sum_{k=n-j+1}^{n} d_k = \sum_{i=1}^{j} d_{n-i+1}.$$

This shows $(\alpha_1, \ldots, \alpha_n) \in Q_n$.

Inclusion $Q_n \subseteq P_1 + \cdots + P_n$. The strategy is to construct the decomposition by greedily moving mass from the last coordinates of α into P_n (which can absorb up to d_n units of total mass), while keeping the prefix $(\alpha'_1, \ldots, \alpha'_{n-1}, 0)$ inside Q_{n-1}. We proceed by induction on n. For $n = 1$, clearly $P_1 = Q_1$.

Assume $n \geq 2$ and $P_1 + \cdots + P_{n-1} = Q_{n-1}$, where Q_{n-1} is the set attached to the truncated sequence $(d_1, \ldots, d_{n-1})$, viewed as a subset of $\mathbb{Z}^{n-1}_{\geq 0} \times \{0\}$. (Note

that the statement of the lemma makes no assumption on the ordering of the d_i, so applying it to the truncated sequence is legitimate and the induction is not circular.) Let $(\alpha_1, \dots, \alpha_n) \in Q_n$. We construct a decomposition

$$(\alpha_1, \dots, \alpha_n) = (\alpha'_1, \dots, \alpha'_{n-1}, 0) + (\alpha''_1, \dots, \alpha''_n)$$

with $(\alpha'_1, \dots, \alpha'_{n-1}, 0) \in Q_{n-1}$ and $(\alpha''_1, \dots, \alpha''_n) \in P_n$.

We define the decomposition recursively for $j = 1, \dots, n$. At step j, if

$$\alpha_{n-j+1} + \sum_{\ell=n-j+2}^{n-1} \alpha'_\ell \le \sum_{i=2}^{j} d_{n-i+1}$$

(with the convention that the sums $\sum_{\ell=n-j+2}^{n-1} \alpha'_\ell$ and $\sum_{i=2}^{j} d_{n-i+1}$ are empty—and therefore equal to 0—when $j = 1$), set $\alpha'_{n-j+1} := \alpha_{n-j+1}$, $\alpha''_{n-j+1} := 0$. Otherwise, set

$$\alpha'_{n-j+1} := \sum_{i=2}^{j} d_{n-i+1} - \sum_{\ell=n-j+2}^{n-1} \alpha'_\ell, \quad \alpha''_{n-j+1} := \alpha_{n-j+1} - \alpha'_{n-j+1}.$$

Here $\sum_{i=2}^{j} d_{n-i+1} = \sum_{k=n-j+1}^{n-1} d_k$ is precisely the budget that Q_{n-1} allows for the suffix starting at index $n - j + 1$; note also that this budget increases by $d_{n-j+1} \ge 0$ from step $j-1$ to step j, so $\alpha'_{n-j+1} \ge 0$ in the second case. At $j = 1$ the budget is the empty sum 0, so the step forces $\alpha'_n = 0$.

In both cases,

$$\sum_{\ell=n-j+1}^{n-1} \alpha'_\ell \le \sum_{i=2}^{j} d_{n-i+1},$$

which establishes $(\alpha'_1, \dots, \alpha'_{n-1}, 0) \in Q_{n-1}$.

To verify $(\alpha''_1, \dots, \alpha''_n) \in P_n$, we need $\sum_{\ell=1}^{n} \alpha''_\ell \le d_n$. If the second case never occurs, then $\alpha'' = 0$ and there is nothing to prove. Otherwise, let j be the *largest* index at which the second case occurs; then $\alpha''_\ell = 0$ for every $\ell < n - j + 1$, and since the second case saturates the budget at step j we obtain

$$\begin{aligned}\sum_{\ell=1}^{n} \alpha''_\ell = \sum_{\ell=n-j+1}^{n} \alpha''_\ell &= \alpha_{n-j+1} - \left(\sum_{i=2}^{j} d_{n-i+1} - \sum_{\ell=n-j+2}^{n-1} \alpha'_\ell \right) + \sum_{\ell=n-j+2}^{n} \alpha''_\ell \\ &= \sum_{\ell=n-j+1}^{n} \alpha_\ell - \sum_{i=2}^{j} d_{n-i+1} \le \sum_{i=1}^{j} d_{n-i+1} - \sum_{i=2}^{j} d_{n-i+1} = d_n,\end{aligned}$$

where the inequality follows from the definition of Q_n.

By the induction hypothesis,

$$(\alpha_1, \dots, \alpha_n) \in Q_{n-1} + P_n \subseteq P_1 + \cdots + P_n.$$

Conclusion. Combining both inclusions, we obtain

$$\mathrm{Supp}(I_{d_1, d_2, \dots, d_n}) = P_1 + \cdots + P_n = Q_n.$$

establishing the claimed characterization. □

C Experimental Validation for Dixon Matrix Size Bound

We provide numerical comparisons between our Fuss–Catalan bound (FC Bound, Corollary 1(2)), Unrestricted bound (Corollary 1(1)), the volume comparison bound (5), motivated by [53, Theorem 4.3] and verified in Section 3.1, and the trivial variable-based bound (4). Table 7 shows the bounds alongside the actual Dixon matrix sizes computed from randomly generated dense $(n; d)$-systems over $\mathbb{F}_p$ with $p = 2^{31} - 159$; generic attainment is subject to the characteristic hypothesis in Lemma 4, and we record the field used for reproducibility. The "True Size" column reports the actual row/column dimension of the Dixon matrix observed in our experiments. For each pair (n, d) we sampled 20 independent, fully dense systems of n equations, all of total degree d, with all coefficients drawn uniformly and independently from $\mathbb{F}_p$; the observed size agreed across all samples. It is the support dimension $D(\mathcal{F})$, not $\operatorname{rank} \mathcal{M}$ (Remark 5).

Table 7. Comparison of Dixon Matrix Size Bounds

n	d	FC Bound	Unrestricted Bound	Volume Bound	Variable Bound	True Size
3	2	5	6	6	8	5
3	3	12	15	13	18	12
3	4	22	28	24	32	22
3	5	35	45	37	50	35
4	2	14	20	21	48	14
4	3	55	84	72	162	55
4	4	140	220	170	384	140
4	5	285	455	333	750	285
5	2	42	70	83	384	42
5	3	273	495	421	1944	273
5	4	969	1820	1333	6144	969
5	5	2530	4845	3255	15000	2530
6	2	132	252	345	3840	132
6	3	1428	3003	2624	29160	1428
6	4	7084	15504	11059	122880	7084
6	5	23751	53130	33750	375000	23751
7	2	429	924	1493	46080	429
7	3	7752	18564	17017	524880	7752
8	2	1430	3432	6657	645120	1430
9	2	4862	12870	30368	10321920	4862
10	2	16796	48620	141093	185794560	16796

D Detailed Derivation of Complexity Ratios in Table 3

This appendix gives the step-by-step derivation of the "Ratio to Dixon" column in Table 3, for $n \geq 2$ fixed and $d \to \infty$. In the ratios below, $\widehat{\mathcal{T}}_M$ denotes the expression inside the displayed cost bound for method M, with logarithmic factors omitted.

For a well-determined $(n; d)$-system, we use the Dixon resultant complexity as the baseline:

$$\mathcal{T}_{\text{Dixon}} = \widetilde{\mathcal{O}}\left(nd \left(\frac{1}{n(d-1)+1} \binom{nd}{n} \right)^{\omega} \right).$$

Gröbner Basis Classical [10] The complexity is

$$\mathcal{T}_{\text{GB}} = \mathcal{O}\left(\binom{n + d_{\text{reg}}}{d_{\text{reg}}}^{\omega} \right).$$

With $d_{\text{reg}} = n(d-1)+1$, this becomes

$$\mathcal{T}_{\text{GB}} = \mathcal{O}\left(\binom{nd+1}{n}^{\omega} \right).$$

Using the binomial identity

$$\binom{nd+1}{n} = \frac{nd+1}{nd-n+1} \binom{nd}{n},$$

we compute the ratio:

$$\begin{aligned} \frac{\widehat{\mathcal{T}}_{\text{GB}}}{\widehat{\mathcal{T}}_{\text{Dixon}}} &= \frac{\binom{nd+1}{n}^{\omega}}{nd \left(\frac{1}{n(d-1)+1} \binom{nd}{n} \right)^{\omega}} \\ &= \frac{(nd+1)^{\omega}}{nd} = (nd)^{\omega-1} \left(1 + \frac{1}{nd} \right)^{\omega} \\ &\sim (nd)^{\omega-1}. \end{aligned}$$

Gröbner Basis F5 Bound [11,15] The complexity is

$$\mathcal{T}_{\text{F5}} = \mathcal{O}\left(n\, d_{\text{reg}} \binom{n + d_{\text{reg}} - 1}{d_{\text{reg}}}^{\omega} \right).$$

Substituting $d_{\text{reg}} = n(d-1)+1$ gives

$$\mathcal{T}_{\text{F5}} = \mathcal{O}\left(n\big(n(d-1)+1\big) \binom{nd}{n-1}^{\omega} \right).$$

Using

$$\binom{nd}{n-1} = \frac{n}{nd-n+1} \binom{nd}{n},$$

the ratio becomes

$$\begin{aligned}\frac{\widehat{\mathcal{T}}_{\mathrm{F5}}}{\widehat{\mathcal{T}}_{\mathrm{Dixon}}} &= \frac{n\big(n(d-1)+1\big)\binom{nd}{n-1}^{\omega}}{nd\left(\frac{1}{n(d-1)+1}\binom{nd}{n}\right)^{\omega}} \\ &= \frac{n^{\omega+1}(nd-n+1)}{nd} \\ &= n^{\omega+1}\left(1-\frac{1}{d}+\frac{1}{nd}\right) \sim n^{\omega+1}.\end{aligned}$$

Gröbner Basis F5 Refined [11] The bound comes from the Macaulay matrix at degree d_{reg}. It has $r = n\binom{n+d_{\mathrm{reg}}-d}{n}$ rows and $c = \binom{n+d_{\mathrm{reg}}}{n}$ columns. Its row echelon form can be computed in $O(rc\varrho^{\omega-2})$ operations, where $\varrho \le c$ is the rank [73]. Thus

$$\mathcal{T}_{\mathrm{F5P}} = \mathcal{O}\left(n\binom{n+d_{\mathrm{reg}}-d}{n}\binom{n+d_{\mathrm{reg}}}{n}^{\omega-1}\right).$$

After substituting $d_{\mathrm{reg}} = n(d-1)+1$, this becomes

$$\mathcal{T}_{\mathrm{F5P}} = \mathcal{O}\left(n\binom{(n-1)d+1}{n}\binom{nd+1}{n}^{\omega-1}\right).$$

Applying the same binomial identity as above gives

$$\begin{aligned}\frac{\widehat{\mathcal{T}}_{\mathrm{F5P}}}{\widehat{\mathcal{T}}_{\mathrm{Dixon}}} &= \frac{n\binom{nd-d+1}{n}\binom{nd+1}{n}^{\omega-1}}{nd\left(\frac{1}{n(d-1)+1}\binom{nd}{n}\right)^{\omega}} \\ &= \frac{\binom{nd-d+1}{n}}{\binom{nd}{n}}\frac{nd-n+1}{d}(nd+1)^{\omega-1}.\end{aligned}$$

For n fixed and $d \to \infty$,

$$\frac{\binom{nd-d+1}{n}}{\binom{nd}{n}} = \prod_{i=0}^{n-1}\frac{nd-d+1-i}{nd-i} \longrightarrow \left(1-\frac{1}{n}\right)^{n}.$$

Consequently,

$$\frac{\widehat{\mathcal{T}}_{\mathrm{F5P}}}{\widehat{\mathcal{T}}_{\mathrm{Dixon}}} \sim \left(1-\frac{1}{n}\right)^{n} n^{\omega}d^{\omega-1} = \Theta\big(n^{\omega}d^{\omega-1}\big),$$

where $1/4 \le (1-1/n)^n < e^{-1}$ for $n \ge 2$. The three Gröbner ratio rows also retain these orders for other growth directions: the binomial quotient above is between $(1-1/n)^n$ and 1 for all $n, d \ge 2$.

FGLM [35,57] The complexity estimate, excluding construction of the multiplication matrices, is

$$\mathcal{T}_{\mathrm{FGLM}} = \mathcal{O}(n\, d_{\mathcal{I}}^{\omega}) = \mathcal{O}(n\, d^{n\omega}).$$

For n fixed and $d \to \infty$, the Fuss–Catalan formula gives

$$C_n^{(d)} = \frac{1}{n!}\prod_{j=0}^{n-2}(nd-j) = \frac{n^{n-1}}{n!}d^{n-1}\big(1+O_n(d^{-1})\big). \tag{34}$$

Substituting this expansion into the ratio yields

$$\begin{aligned}\frac{\widehat{\mathcal{T}}_{\mathrm{FGLM}}}{\widehat{\mathcal{T}}_{\mathrm{Dixon}}} &= \frac{n\, d^{n\omega}}{nd\left(C_n^{(d)}\right)^{\omega}} \\ &\sim \frac{n\, d^{n\omega}}{nd\left(\frac{n^{n-1}}{n!}\right)^{\omega} d^{(n-1)\omega}} \\ &= \left(\frac{n!}{n^{n-1}}\right)^{\omega} d^{\omega-1}.\end{aligned}$$

Stirling bounds simplify the factorial coefficient to

$$\left(\frac{n!}{n^{n-1}}\right)^{\omega} = \Theta\left(n^{3\omega/2}e^{-n\omega}\right), \tag{35}$$

with constants depending only on ω. Hence the ratio in the table is

$$\frac{\widehat{\mathcal{T}}_{\mathrm{FGLM}}}{\widehat{\mathcal{T}}_{\mathrm{Dixon}}} = \Theta\left(n^{3\omega/2}d^{\omega-1}e^{-n\omega}\right).$$

The factorial expression is the exact leading coefficient for fixed n; the $e^{-n\omega}$ form displays its order of magnitude as a function of n. The high-degree expansion (34) is not a uniform joint limit in (n, d).

BNS Change of Order [17] Under the shape and stability assumptions of [17, Theorem 1.1], the complexity is

$$\mathcal{T}_{\mathrm{BNS}} = \widetilde{\mathcal{O}}\left(t^{\omega-1}d_{\mathcal{I}}\right).$$

Here t counts the grevlex basis elements whose leading monomial is divisible by the smallest variable. For the generic-system estimate, use $d_{\mathcal{I}} = d^n$ and $t = \Theta(d^{n-1}/\sqrt{n})$ as in [17, Section 6.2 and Table 1]. Substitution gives

$$\begin{aligned}\mathcal{T}_{\mathrm{BNS}} &= \widetilde{\mathcal{O}}\left(\left(d^{n-1}n^{-1/2}\right)^{\omega-1}d^{n}\right) \\ &= \widetilde{\mathcal{O}}\left(n^{-(\omega-1)/2}d^{n\omega-\omega+1}\right).\end{aligned}$$

Using (34) and then (35), the ratio becomes

$$\begin{aligned}\frac{\widehat{\mathcal{T}}_{\mathrm{BNS}}}{\widehat{\mathcal{T}}_{\mathrm{Dixon}}} &= \frac{t^{\omega-1}d^n}{nd\left(C_n^{(d)}\right)^{\omega}} \\ &= \Theta\left(\left(\frac{n!}{n^{n-1}}\right)^{\omega} n^{-(\omega+1)/2}\right) \\ &= \Theta\left(n^{\omega-1/2}e^{-n\omega}\right).\end{aligned}$$

BNS takes the grevlex basis as input and reads its polynomial-matrix representation directly under the stability assumptions.

Other asymptotic directions. For $d \geq 2$ fixed and $n \to \infty$, equation (39) instead gives $C_n^{(d)} = \Theta_d(\gamma_d^n/n^{3/2})$. Writing $\eta_d = d/\gamma_d = ((d-1)/d)^{d-1}$, we obtain

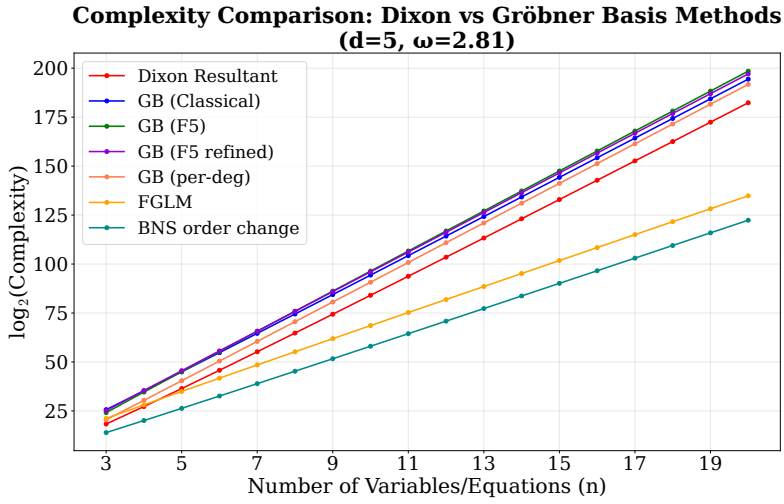
$$\frac{\widehat{\mathcal{T}}_{\mathrm{FGLM}}}{\widehat{\mathcal{T}}_{\mathrm{Dixon}}} = \Theta_d\left(n^{3\omega/2}\eta_d^{n\omega}\right), \qquad \frac{\widehat{\mathcal{T}}_{\mathrm{BNS}}}{\widehat{\mathcal{T}}_{\mathrm{Dixon}}} = \Theta_d\left(n^{\omega-1/2}\eta_d^{n\omega}\right).$$

These change-of-order ratios decrease exponentially in this regime. The high-degree simplifications in Table 3 therefore require the stated growth direction; the plots use the unsimplified cost expressions.

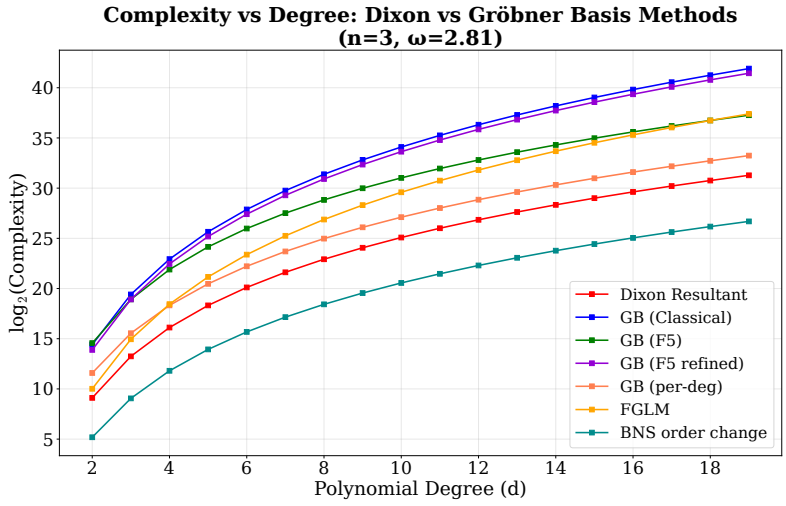
Other elimination algorithms. For two eliminated variables, structured Sylvester-resultant algorithms such as Villard's [79] have a better asymptotic complexity, even though the Dixon construction works with a smaller matrix. On the AO systems of Section 5, the relevant Gröbner competitors also include FreeLunch [14] and CheapLunch [7], for which the grevlex basis is essentially free and change of order dominates. Their comparison with Dixon therefore depends on the number of variables and the degrees; the full F4/F5-plus-change-of-order model does not describe this special structure.

E Further Comparative Plots

We present a more comprehensive comparison of the computational complexity of the Dixon resultant and Gröbner basis methods from multiple perspectives.

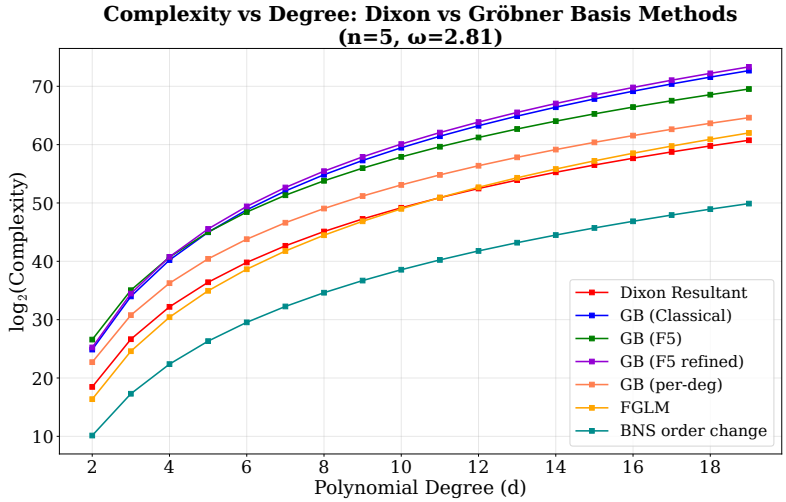


(a)

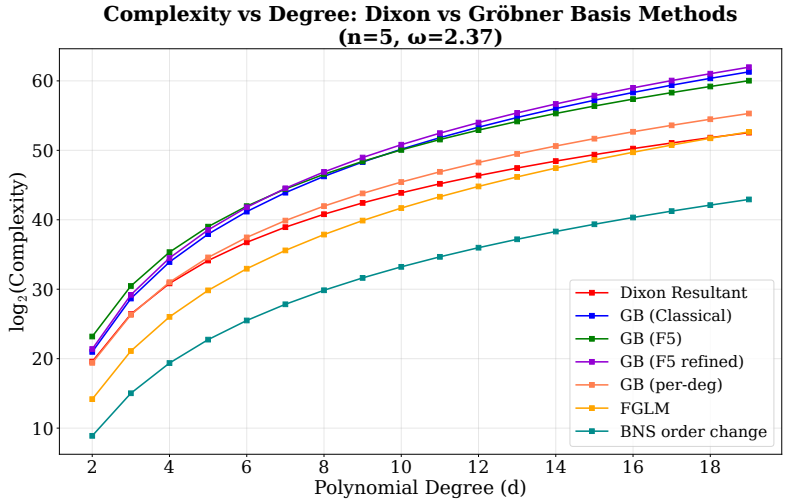


(b)

Fig. 6. Complexity comparison between the Dixon resultant and Gröbner basis



(c)



(d)

Fig. 6 (continued). Complexity comparison between the Dixon resultant and Gröbner basis.

F Degree-Aware Submatrix Selection Algorithms

Algorithm 1 gives the pseudocode for the selection strategy described in Section 4.1.

Algorithm 1 Degree-aware maximal-rank submatrix selection

Input: Polynomial matrix $\mathcal{M} \in \mathbb{K}[t_1, \ldots, t_m]^{r \times c}$ (possibly rectangular or rank-deficient)
Output: Row indices $\mathcal{R}$ and column indices $\mathcal{C}$ with $|\mathcal{R}| = |\mathcal{C}|$ such that $\mathcal{M}[\mathcal{R}, \mathcal{C}]$ has maximal rank and minimal degree estimate
Initialize $\mathcal{R} \leftarrow \emptyset$, $\mathcal{C} \leftarrow \emptyset$
Choose concrete evaluation point $\alpha = (\alpha_1, \ldots, \alpha_m) \in \mathbb{F}^m$
Evaluate $\mathcal{M}_{\text{const}} \leftarrow \mathcal{M}(\alpha_1, \ldots, \alpha_m)$ {Constant matrix}
Compute row degree weights: $d_r(i) \leftarrow \max_j \deg(\mathcal{M}_{i,j})$ for each row i
Compute column degree weights: $d_c(j) \leftarrow \max_i \deg(\mathcal{M}_{i,j})$ for each column j
Sort row indices by increasing degree weights: π_r
Sort column indices by increasing degree weights: π_c
Initialize basis tracker: $\mathcal{B} \leftarrow \emptyset$
for each candidate index k in π_r **do**
    Extract row vector $\boldsymbol{v}_k$ from $\mathcal{M}_{\text{const}}$
    Eliminate $\boldsymbol{v}_k$ against existing basis vectors in $\mathcal{B}$
    if $\boldsymbol{v}_k \neq \mathbf{0}$ after elimination **then**
        Normalize $\boldsymbol{v}_k$ and add it to $\mathcal{B}$
        Include k in $\mathcal{R}$
    end if
end for
Reset basis tracker: $\mathcal{B} \leftarrow \emptyset$
for each candidate index k in π_c **do**
    Extract column vector $\boldsymbol{v}_k$ from $\mathcal{M}_{\text{const}}$
    Eliminate $\boldsymbol{v}_k$ against existing basis vectors in $\mathcal{B}$
    if $\boldsymbol{v}_k \neq \mathbf{0}$ after elimination **then**
        Normalize $\boldsymbol{v}_k$ and add it to $\mathcal{B}$
        Include k in $\mathcal{C}$
    end if
end for
return $\mathcal{R}, \mathcal{C}$

It is straightforward to observe that the computational cost of our algorithm essentially reduces to two Gaussian eliminations on constant matrices: one for row selection and the other for column selection. Compared with the complexity in Equation (15), this step incurs no additional asymptotic overhead.

G Experimental Results of Degree-Aware Submatrix Selection

We present the experimental results of Algorithm 1 to validate Heuristic 2. Table 8 compares the performance of our degree-aware selection strategy against naive random selection across various polynomial system configurations.

Table 8. Frequency of Bézout-bound violations

n_x	k	Degrees $(d_1, \ldots, d_{n_x+1})$	Bézout Bound $\prod d_i$	Bézout violations	
				Naive	Algorithm 1
2	1	$(3, 4, 5)$	60	195/200	0/200
2	1	$(5, 6, 6)$	180	200/200	0/200
2	1	$(5, 5, 5)$	125	192/200	0/200
5	1	$(2, 2, 2, 2, 2, 2)$	64	199/200	0/200
3	2	$(2, 2, 3, 3)$	36	177/200	1/200
Observed violation frequency				963/1000	1/1000

In Table 8, n_x denotes the number of variables to eliminate (so the system contains $n_x + 1$ polynomials, consistent with the convention in the main text where n is the number of polynomials), k denotes the number of parameters retained. We count cases with $\deg(\mathrm{Res}(\mathcal{F})) > \prod d_i$; passing this test does not certify the absence of extraneous factors or excess multiplicities. The naive method uses random submatrix selection. All experiments were conducted over $\mathbb{F}_{65537}$ with 200 repetitions per system configuration. The observed violations for Algorithm 1 exceeded the Bézout bound by at most 1–2.

H Detailed Stage-wise Complexity Parameters

This appendix provides the detailed parameter derivations and the full set of five determinant models (expansion, Bareiss, Kronecker+HNF, interpolation, sparse) for the two symbolic-determinant stages whose preferred models were summarized in Section 3.2. We keep the notation of Section 3.2 throughout: $\mathcal{F}$ is an $(n; d)$-system in $n - 1 + k$ variables, $C_n^{(d)}$ is the generic Dixon-matrix size, and we use the determinant-method complexity templates from Appendix I.

H.1 Step 1 Parameter Derivations

This subsection derives the four abstract parameters $(\mu_1, S_1, L_1, \delta_1)$ and the additional grid-size parameter N_1 used to instantiate the five Step 1 complexity models. The expressions for (μ_1, S_1, L_1) recover (12) in Section 3.2, and substituting them into the determinant templates of Table 2 reproduces (13)–(14).

The refined construction of the cancellation matrix from (3) lowers several *dual*-variable degrees by one through the difference quotients in $x_i - \bar{x}_i$, but this affects only the eliminated and dual variables and not the retained parameters $\boldsymbol{a}$. For the asymptotic estimates below we use uniform upper bounds that are simpler to state. The determinant in Step 1 involves $\ell = 2n+k-2$ variables (the $n-1$ original variables, the $n-1$ dual variables, and the k retained parameters), with each entry having parameter total degree at most d. Summing across the n rows of the determinant, the parameter total degree of $\det(\widehat{C})$ is therefore at most

$$B_1 := \sum_{i=1}^{n} d = nd.$$

Dense monomial-count proxy. The expansion-by-minors model uses the dense monomial-count proxy

$$\mu_1 \le \binom{n-1+k+d}{n-1+k},$$

i.e. the number of dense monomials in the original f_j when viewed as a polynomial in the $n - 1 + k$ variables of $\boldsymbol{x}$ and $\boldsymbol{a}$ of total degree at most d. Explicitly expanding the difference-quotient entries would introduce a factor $\max\{1, d/(n+k)\}$ in the uniform entry-support bound. The theoretical expansion model avoids this factor by multiplying the undivided numerators of a difference-quotient row by the cached minors, summing the row expansion, and only then dividing exactly by the common linear factor $x_i - \bar{x}_i$. Each numerator has at most $2\mu_1$ terms, and this linear division is soft-linear in the input and output support sizes. Under the stated intermediate-support proxy $U := S_1$, the cost therefore remains $\widetilde{O}(n2^n\mu_1 S_1)$, as in (13). This is an evaluation strategy for the same refined determinant, rather than a claim that expanded difference quotients have at most μ_1 terms.

Variable-wise degree bounds. The Kronecker+HNF model uses the variable-wise degree bounds derived from the refined cancellation-matrix structure. A row-by-row analysis of $\widehat{C}$ (the variable x_i appears in rows $1, \dots, i$ with degree $\le d$, in row $i+1$ with degree $\le d-1$ after the difference quotient, and is absent from rows $i+2, \dots, n$ where it has been replaced by $\bar{x}_i$; an analogous count applies to $\bar{x}_i$) gives

$$b_{x_i} := \deg_{x_i}(\det \widehat{C}) \le (i+1)d - 1, \qquad b_{\bar{x}_i} := \deg_{\bar{x}_i}(\det \widehat{C}) \le (n-i)d - 1,$$
$$b_{a_j} := \deg_{a_j}(\det \widehat{C}) \le nd,$$

for $i = 1, \dots, n-1$ and $j = 1, \dots, k$. Every entry has total degree at most d, so in particular $e_{x_i}, e_{\bar{x}_i}, e_{a_j} \le d$. Kronecker substitution must use the determinant-level base $b_* + 1$ (per Appendix I). For concreteness, let $b_1, \dots, b_\ell$ denote the variable-wise determinant degrees in some fixed ordering of the $\ell = 2n + k - 2$ variables. The largest Kronecker weight is $\prod_{i=1}^{\ell-1}(b_i + 1)$; using the entry total-degree bound, the average column degree of the univariate matrix is bounded by

$$\delta_1 := d \cdot \prod_{i=1}^{\ell-1}(b_i + 1) \;\le\; \prod_{i=1}^{\ell}(b_i + 1),$$

where d is the total-degree bound of an entry and $d \le b_\ell + 1$ for the structural bounds above. The estimate is valid for any fixed variable ordering, although its value may depend on that ordering. The HNF cost is then $T_1^{\mathrm{HNF}} = \widetilde{O}(n^\omega \delta_1)$.

Interpolation parameters. For evaluation-interpolation, the tensor-grid size is

$$N_1 = \prod_{i=1}^{2n+k-2} (b_i + 1),$$

and one probe evaluates the n^2 entries of $\widehat{C}$ at a point of $\mathbb{F}_q^\ell$ and computes the determinant of the resulting $n \times n$ constant matrix. Each entry f_j is a polynomial in the $n - 1 + k$ variables of $\boldsymbol{x}, \boldsymbol{a}$ with at most μ_1 monomials, so building the evaluated matrix from n underlying polynomials at n points costs $\widetilde{O}(n^2 \mu_1)$ field operations, and the constant determinant costs $\widetilde{O}(n^\omega)$. Therefore

$$L_1 = \widetilde{O}(n^\omega + n^2 \mu_1).$$

Sparse term bound. The sparse model uses the structural term bound

$$S_1 \le \left(C_n^{(d)}\right)^2 \binom{B_1 + k}{k},$$

where the factor $\left(C_n^{(d)}\right)^2$ comes from the bilinear Dixon support (each of $\boldsymbol{x}, \bar{\boldsymbol{x}}$ contributes at most $C_n^{(d)}$ distinct monomials by Lemma 4), and $\binom{B_1+k}{k}$ bounds the number of parameter monomials of total degree at most B_1 in the k retained parameters $\boldsymbol{a}$; see Appendix I for the full derivation.

Step 1 complexity models. Substituting these parameters into the determinant models of Table 2 yields the following five Step 1 complexity estimates (all in field operations over $\mathbb{F}_q$):

$$\begin{aligned} T_1^{\text{minor}} &= \widetilde{O}(n2^n\mu_1 S_1), \\ T_1^{\text{Bareiss}} &= \widetilde{O}(n^3 S_1^2), \\ T_1^{\text{HNF}} &= \widetilde{O}(n^\omega \delta_1), \\ T_1^{\text{eval}} &= \widetilde{O}(N_1(L_1 + \ell B_1)), \\ T_1^{\text{sparse}} &= \widetilde{O}(L_1 S_1 + S_1 \log q). \end{aligned}$$

In the expansion and Bareiss models we use S_1 as the support proxy U for intermediate minors, justified by the genericity argument of Appendix I.

FDixon recursive construction. Besides the five determinant models, the direct recursive construction of Zhao and Fu [82] builds the Dixon matrix *without* first forming the $n \times n$ cancellation determinant, so it replaces Steps 1 and 2 by a single block-recursive construction. Following the complexity model of Qin et al. [66, Theorem 3.1], our implementation uses the surrogate

$$T_2^{\text{FDixon}} = \widetilde{O}\left(m_1^2 \, (n!)^3 \prod_{i=2}^{n} m_i^3 \right),$$

where the sorted surrogate degrees m_i are the per-variable degree bounds selected in the code. (One can verify the $(n!)^3$ factor by combining the dominant term $n^3 \cdot ((n-1)!)^3$ of [66, Eq. (19)–(20)] with $n \cdot (n-1)! = n!$.) The factorial factor reflects the recursive enumeration of monomial supports inside the block recursion, which makes the estimate most attractive when n is small and the degrees m_i are large; for systems with many eliminated variables it becomes less competitive than the direct Step 1 path.

H.2 Step 4 Parameter Derivations

This subsection derives the abstract Step 4 parameters $(D_4, N_4, L_4, \delta_4, \mu_4, S_4)$ used to instantiate the five Step 4 complexity models. The expressions for $(D_4, N_4, L_4, \delta_4)$ recover (16) in Section 3.2, and substituting them into the determinant templates of Table 2 reproduces (17)–(18).

Step 4 computes a symbolic determinant of a $C_n^{(d)} \times C_n^{(d)}$ polynomial matrix in the k retained parameters $\boldsymbol{a}$. Let $E_4 \le nd$ be an entry-wise parameter-degree bound for the Step 4 matrix and $D_4 \le d^n$ be the Bézout total-degree bound for the elimination polynomial.

Parameter expressions. Kronecker substitution must use a base $\ge C_n^{(d)} E_4 + 1$ to avoid overflow in the encoded determinant; the resulting univariate matrix has

average column degree bounded by

$$\delta_4 := E_4 \prod_{j=1}^{k-1} (C_n^{(d)} E_4 + 1),$$

which collapses to $\delta_4 = E_4 \le nd$ in the well-determined case $k = 1$. The evaluation-interpolation and sparse-interpolation parameters are

$$N_4 \le (D_4 + 1)^k, \qquad S_4 \le \binom{D_4 + k}{k}, \qquad L_4 = \widetilde{O}\Big(\big(C_n^{(d)}\big)^{\omega} \Big).$$

The dense monomial-count proxy for one matrix entry is

$$\mu_4 \le \binom{k + E_4}{k}.$$

Step 4 complexity models. The full set of five determinant models (all in field operations over $\mathbb{F}_q$) is:

$$\begin{aligned}
T_4^{\text{minor}} &= \widetilde{O}\Big(C_n^{(d)} 2^{C_n^{(d)}} \mu_4 S_4 \Big), \\
T_4^{\text{Bareiss}} &= \widetilde{O}\Big(\big(C_n^{(d)}\big)^3 S_4^2 \Big), \\
T_4^{\text{HNF}} &= \widetilde{O}\Big(\big(C_n^{(d)}\big)^{\omega} \delta_4 \Big), \\
T_4^{\text{eval}} &= \widetilde{O}(N_4 (L_4 + k D_4)), \\
T_4^{\text{sparse}} &= \widetilde{O}(L_4 S_4 + S_4 \log q).
\end{aligned}$$

Full table of stage-wise models. Table 9 collects all the formulas above.

Table 9. Complexity models instantiated for the first and final symbolic determinants.

Method	**Step 1/2**	**Step 4**
Expansion	$\widetilde{O}(n 2^n \mu_1 S_1)$	$\widetilde{O}\Big(C_n^{(d)} 2^{C_n^{(d)}} \mu_4 S_4 \Big)$
Bareiss	$\widetilde{O}(n^3 S_1^2)$	$\widetilde{O}\Big(\big(C_n^{(d)}\big)^3 S_4^2 \Big)$
Kronecker	$\widetilde{O}(n^{\omega} \delta_1)$	$\widetilde{O}\Big(\big(C_n^{(d)}\big)^{\omega} \delta_4 \Big)$
Interpolation	$\widetilde{O}(N_1 (L_1 + \ell B_1))$	$\widetilde{O}(N_4 (L_4 + k D_4))$
Sparse	$\widetilde{O}(L_1 S_1 + S_1 \log q)$	$\widetilde{O}(L_4 S_4 + S_4 \log q)$
FDixon	$\widetilde{O}(m_1^2 (n!)^3 \prod_{i=2}^{n} m_i^3)$	—

H.3 Step 1 vs Step 4 Comparison

This subsection gives the detailed comparison behind the main-text statement that the symbolic cost is usually governed by the final determinant computation. We first compare the most favorable first-determinant estimate with the univariate HNF model for the final determinant, and then relate this theoretical picture to the trend plots used in the paper.

Notation. We keep the notation of Section 3.2 and the preceding subsections. In particular, the first determinant is modeled by

$$T_1^{\text{minor}}, \quad T_1^{\text{Bareiss}}, \quad T_1^{\text{HNF}}, \quad T_1^{\text{eval}}, \quad T_1^{\text{sparse}}, \quad T_2^{\text{FDixon}},$$

while the final determinant is modeled by

$$T_4^{\text{minor}}, \quad T_4^{\text{Bareiss}}, \quad T_4^{\text{HNF}}, \quad T_4^{\text{eval}}, \quad T_4^{\text{sparse}}.$$

Comparison of T_1^{sparse} and T_4^{HNF} as n grows. Assume the well-determined setting $k = 1$ and keep d fixed. Then

$$T_1^{\text{sparse}} = \widetilde{O}(L_1 S_1 + S_1 \log q) = \widetilde{O}\Big(\big(C_n^{(d)}\big)^2 \operatorname{poly}(n)\Big),$$

because $S_1 \le \big(C_n^{(d)}\big)^2 (B_1+1) = \widetilde{O}\Big(\big(C_n^{(d)}\big)^2\Big)$ for $k = 1$, and one probe costs $L_1 = \widetilde{O}(n^\omega + n^2 \mu_1)$ with $\mu_1 \le \binom{n+d}{n}$, hence $L_1 = \widetilde{O}(\operatorname{poly}(n))$ for fixed d (the $S_1 \log q$ term is also absorbed by $\widetilde{O}\Big(\big(C_n^{(d)}\big)^2 \operatorname{poly}(n)\Big)$ since $\log q$ is polylogarithmic in the problem size). On the other hand,

$$T_4^{\text{HNF}} = \widetilde{O}\Big(nd\,\big(C_n^{(d)}\big)^\omega\Big).$$

For fixed d, the Fuss–Catalan quantity satisfies $C_n^{(d)} = \Theta(\gamma_d^n / n^{3/2})$ for a constant $\gamma_d > 1$. Hence T_1^{sparse} grows essentially like $\big(C_n^{(d)}\big)^2$ times a polynomial factor, while T_4^{HNF} grows like $\big(C_n^{(d)}\big)^\omega$ times a polynomial factor. For $\omega > 2$ (which is the regime of all currently known fast linear-algebra algorithms, including the practically achievable $\omega \approx 2.81$ used in our timing tables), the dominant $C_n^{(d)}$-exponent of the final determinant is strictly larger, so the growth of T_4^{HNF} is asymptotically faster than that of T_1^{sparse} as n increases. The borderline case $\omega = 2$ makes the two $C_n^{(d)}$-exponents equal, so the two stages are of the same asymptotic order in $C_n^{(d)}$ and their relative dominance is determined by the polynomial-in-(n, d) prefactors and implementation constants rather than by the asymptotics; this is precisely why we distinguish the two regimes.

Comparison of T_1^{HNF} and T_4^{HNF} as d grows. Fix n and again take $k = 1$. The variable-wise determinant degrees in the first determinant satisfy $b_i = \Theta(nd)$,

with $\ell = 2n-1$, and per-entry degree $\Theta(d)$, so (treating n as fixed and letting $d \to \infty$)

$$\delta_1 = d \cdot \prod_{i=1}^{\ell-1}(b_i+1) = \Theta(d^{2n-1}), \qquad T_1^{\mathrm{HNF}} = \widetilde{O}(n^\omega d^{2n-1}).$$

For the final determinant,

$$C_n^{(d)} = \Theta(d^{n-1}), \qquad T_4^{\mathrm{HNF}} = \widetilde{O}\Big(nd\left(C_n^{(d)}\right)^\omega\Big) = \widetilde{O}(d^{\omega(n-1)+1}).$$

Since $\omega > 2$, we have $\omega(n-1)+1 > 2n-1$ for every $n \geq 2$. Thus, for fixed n, the HNF complexity estimate for Step 4 has a strictly larger exponent in d than that for Step 1. At the boundary $\omega = 2$, the exponents coincide, so this comparison alone does not establish which stage dominates; the suppressed polylogarithmic factors and constants may affect their relative costs. In particular, the constants may depend strongly on n, which is held fixed in this analysis. For $\omega > 2$, the comparison supports using Step 4 as the asymptotic bottleneck within the HNF model in the fixed-n, high-degree regime. This is consistent with our experiments, in which the final symbolic determinant dominates the running time. We treat $\omega = 2$ as an idealized boundary used for the uniform-exponent security comparisons.

Consequences for the overall comparison. The two formula comparisons above explain the qualitative picture observed in the implementation and in the plots: Figures 7 and 8 illustrate the two regimes:

1. for low-variable/high-degree systems, the final determinant is clearly the dominant symbolic cost;
2. for high-variable/low-degree systems, the first determinant rises sharply, and sparse interpolation is the only first-determinant model that can stay below the final determinant;
3. the numerical submatrix-selection phase remains below the symbolic final determinant throughout the parameter ranges considered in the paper.

For $\omega > 2$, with this method selection the first determinant never dominates: Kronecker+HNF controls it in the high-degree regime and cached expansion or sparse interpolation controls it in the high-variable regime. The dominance, however, is asymptotic: at moderate (n, d) the polynomial prefactors of the Step-1 models can still exceed the $\left(C_n^{(d)}\right)^{\omega-2}$ gap, and at $\omega = 2$ the conclusion is reversed altogether, since the $\Theta\Big(\left(C_n^{(d)}\right)^2\Big)$ coefficient positions of the Dixon polynomial already match the $C_n^{(d)}$-exponent of Step 4. This is why the $\omega = 2.37$ panel of Figure 6 reports the maximum over the two stages. This matches the measured runtimes, where the final symbolic determinant (Step 4) consumes the overwhelming majority of the time. Accordingly, the main text uses T_4^{best} as the overall symbolic term. The specialization to the well-determined case $k = 1$, including the comparison of T_4^{HNF} with both T_1^{sparse} and T_1^{minor} used in Section 3.2, is treated separately in Appendix H.4.

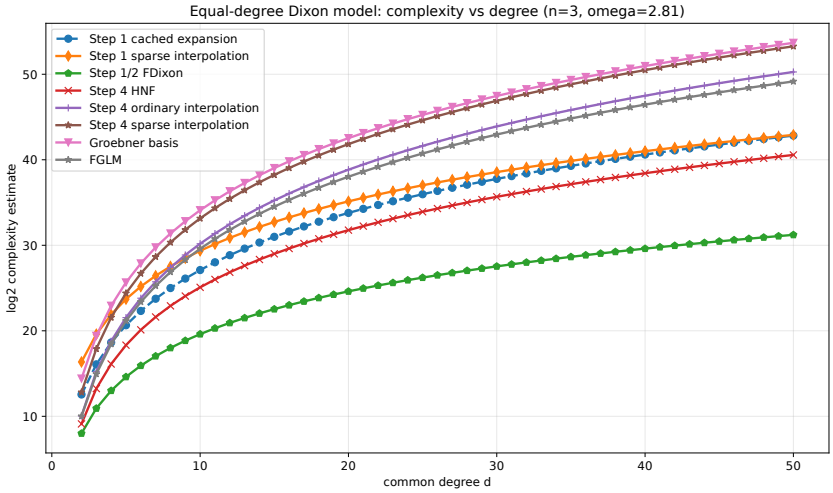


Fig. 7. Comparison of the first and final determinant costs as the polynomial degree varies. The final determinant keeps the largest growth rate in the low-variable/high-degree regime.

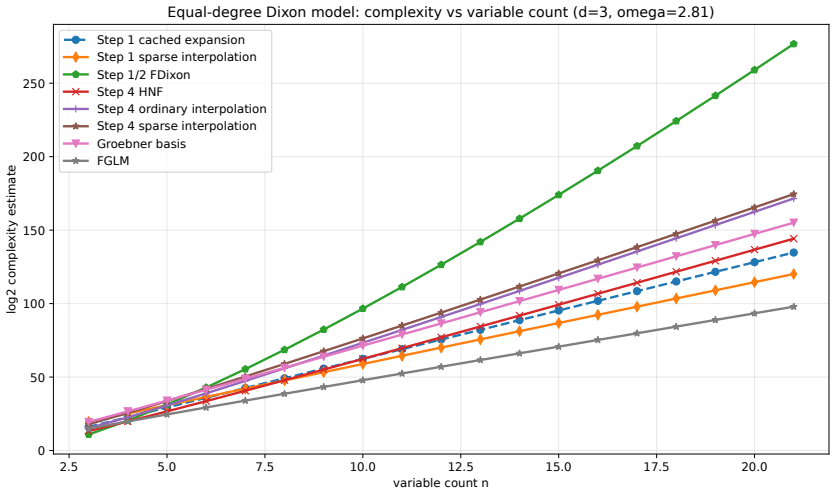


Fig. 8. Comparison of the first and final determinant costs as the number of variables varies. In the low-degree/high-variable regime, sparse interpolation is the only first-determinant model that typically remains below the final determinant.

H.4 Well-determined Case ($k = 1$)

We now specialize the comparison in Appendix H.3 to the well-determined setting $k = 1$, which is the case used both in the comparison with Gröbner basis methods (Section 3.3) and in the AO applications of Section 5. The argument is more explicit than the general case because the parameter space is one-dimensional, so $D_4 \le d^n$, $E_4 \le nd$, $\delta_4 = nd$, and Kronecker+HNF reduces to a single univariate HNF on a $C_n^{(d)} \times C_n^{(d)}$ matrix.

Explicit formulas. Substituting $k = 1$ into the Step 1 and Step 4 templates of Appendices H.1 and H.2 gives

$$T_4^{\mathrm{HNF}} = \widetilde{O}\Big(nd\,\big(C_n^{(d)}\big)^{\omega}\Big), \tag{36}$$

$$T_1^{\mathrm{sparse}} = \widetilde{O}\big(n^2\,(nd+1)\binom{n+d}{n}\big(C_n^{(d)}\big)^2\big) = \widetilde{O}\big(n^3\,d\,\binom{n+d}{n}\big(C_n^{(d)}\big)^2\big), \tag{37}$$

$$T_1^{\mathrm{minor}} = \widetilde{O}\big(n\,2^n(nd+1)\binom{n+d}{n}\big(C_n^{(d)}\big)^2\big) = \widetilde{O}\big(n^2 d\,2^n\binom{n+d}{n}\big(C_n^{(d)}\big)^2\big), \tag{38}$$

where, in (37), the factor $nd+1$ is combined with the n^2 probe-cost factor into the prefactor $n^3 d$ (this is the form used in the main text); the comparisons below are in any case insensitive to this d-factor, since it cancels against the nd factor of (36) when the ratio is formed.

Asymptotic size of $C_n^{(d)}$. For fixed $d \ge 2$ and $n \to \infty$, the Fuss–Catalan formula and Stirling's approximation give

$$C_n^{(d)} \;=\; \frac{1}{n(d-1)+1}\binom{nd}{n} \;=\; \Theta\Big(\frac{\gamma_d^{\,n}}{n^{3/2}}\Big), \qquad \gamma_d := \frac{d^d}{(d-1)^{d-1}}, \tag{39}$$

where $\gamma_d > 1$ for $d \ge 2$. The sequence $(\gamma_d)_{d\ge 2}$ is strictly increasing, with minimum $\gamma_2 = 4$, and grows like $\gamma_d \sim e\,d$ as $d \to \infty$. Equation (39) shows that $C_n^{(d)}$ grows exponentially in n for any fixed $d \ge 2$, while $\binom{n+d}{n} = \Theta(n^d)$ is only polynomial in n for fixed d. This separation is the key quantitative input for the two comparisons below.

Comparison of T_4^{HNF} with T_1^{sparse}. Forming the ratio and cancelling the common d-factors gives

$$\frac{T_4^{\mathrm{HNF}}}{T_1^{\mathrm{sparse}}} \;=\; \widetilde{\Theta}\left(\frac{\big(C_n^{(d)}\big)^{\omega-2}}{n^2\,\binom{n+d}{n}}\right). \tag{40}$$

Substituting (39), the numerator $\big(C_n^{(d)}\big)^{\omega-2}$ equals $\gamma_d^{(\omega-2)n}/n^{3(\omega-2)/2}$, which grows *exponentially* in n as long as $\gamma_d^{\omega-2} > 1$, i.e. as long as $\omega > 2$ (since $\gamma_d > 1$ for $d \ge 2$). The denominator is polynomial in n for fixed d. Hence for every $\omega > 2$ and every fixed $d \ge 2$, the ratio (40) tends to infinity as $n \to \infty$, so

$$T_4^{\mathrm{HNF}} \;\gg\; T_1^{\mathrm{sparse}} \quad \text{asymptotically in } n.$$

Comparison of T_4^{HNF} with T_1^{minor}. The same cancellation yields

$$\frac{T_4^{\mathrm{HNF}}}{T_1^{\mathrm{minor}}} \;=\; \widetilde{\Theta}\!\left(\frac{\left(C_n^{(d)}\right)^{\omega-2}}{n\,2^n\binom{n+d}{n}}\right) \;=\; \widetilde{\Theta}\!\left(\frac{(\gamma_d^{\omega-2}/2)^{\,n}}{\mathrm{poly}(n,d)}\right). \tag{41}$$

This ratio tends to infinity as $n \to \infty$ *iff* the base satisfies

$$\gamma_d^{\omega-2} > 2, \qquad \text{equivalently} \qquad \omega \;>\; 2 + \frac{\log 2}{\log \gamma_d}. \tag{42}$$

The right-hand side of (42) is strictly decreasing in d: for $d = 2$ it equals $2 + \frac{\log 2}{\log 4} = 2.5$; for $d = 3$ it equals $2 + \frac{\log 2}{\log(27/4)} \approx 2.363$; and it tends to 2 from above as $d \to \infty$. In particular, for the Strassen exponent $\omega \approx 2.81$ used in our timing tables and Sections 5–3.3, condition (42) is met for all $d \geq 2$ (e.g. $\gamma_2^{0.81} \approx 3.09 > 2$). Hence

$$T_4^{\mathrm{HNF}} \;\gg\; T_1^{\mathrm{minor}} \quad \text{asymptotically in } n,$$

for every $\omega \geq 2.81$ and every $d \geq 2$ (and more generally whenever (42) holds).

Boundary case $\omega = 2$. At the idealized boundary $\omega = 2$, the $C_n^{(d)}$-exponents of T_4^{HNF} and of both T_1^{sparse} and T_1^{minor} coincide, so the asymptotic dominance is determined by the polynomial-in-(n, d) prefactors rather than by a $C_n^{(d)}$-exponent gap. The value $\omega = 2$ is used in this paper only for `Vision`, for consistency with [62]; there it is applied identically to Dixon and to every competitor, so the comparison should be read as a "uniform-ω ratio" rather than as a literal asymptotic statement about any single method. For `Vision` this is why the reported cost is the maximum over *all* stages of the chain rather than the Step 4 term alone (Appendix K). For any $\omega > 2$ already at infinitesimal margin, the $\left(C_n^{(d)}\right)^{\omega-2}$ factor reactivates and T_4^{HNF} dominates T_1^{sparse}; condition (42) then governs the comparison with T_1^{minor}. The Strassen exponent $\omega \approx 2.81$ used in the timing tables comfortably sits in this regime.

Consequence. With the best applicable construction model, Step 4 is asymptotically dominant for $\omega > 2$ in the regimes analyzed above, and is also the empirical bottleneck. We therefore use $T = T_4^{\mathrm{HNF}}$ as the asymptotic estimate for well-determined systems, as in (19).

I Alternative Determinant Computation Methods

This appendix expands the determinant-method summary from Table 2. We record both the complexity formulas and the stage-specific origin of the parameters used in the comparison code.

Notation. Let $P(\boldsymbol{y})$ be an $m \times m$ polynomial matrix in variables $\boldsymbol{y} = (y_1, \ldots, y_v)$. Let b_i bound the degree of $\det(P)$ in y_i (this determines the Kronecker substitution base, which must avoid overflow), and let $e_i \le b_i$ bound the degree of a single entry of P in y_i. With $c_i = \prod_{j<i}(b_j + 1)$, define

$$\delta = \sum_{i=1}^{v} e_i c_i, \qquad N = \prod_{i=1}^{v} (b_i + 1),$$

where δ bounds the entry degree, hence the average column degree, of the univariate matrix obtained after Kronecker substitution. Since $\sum_{i<v} b_i c_i = c_v - 1$, we have $\delta \le (e_v + 1)c_v - 1 < 2e_v c_v$ when $e_v \ge 1$; this yields the asymptotic cost $\widetilde{O}(m^\omega e_v c_v)$. If every entry has total degree at most E, the bound $\delta = E c_v$ may be used instead. This gives the Step 1 and Step 4 specializations with $E = d$ and $E = E_4 \le nd$, respectively. We also write $B := \max_i b_i$, L for the cost of one determinant probe, S for a sparsity bound on $\det(P)$, U for an upper bound on the support size of intermediate minors arising during a chosen evaluation strategy, μ for a dense monomial-count bound on one entry of P, and $\widetilde{M}(u)$ for soft-Oh multiplication at degree u.

Intermediate-minor support proxy (modeling convention). Several models (expansion, Bareiss) involve intermediate minors of P, and we take $U := S$ for them. This is a *modeling convention*, not a proved inequality: a $j \times j$ minor is in general not a sub-polynomial of $\det(P)$, since the Minkowski sum of j row supports need not be contained in the Minkowski sum of all s row supports (this would require the remaining supports to contain 0), and $\mathrm{Supp}(\det P)$ is itself only contained in, not equal to, that Minkowski sum. What motivates the convention is that under our genericity assumption (Definition 5) every entry of P has full support and no coefficient cancellation occurs, so the support of an intermediate minor is exactly the Minkowski sum of its row supports and is of the same order of magnitude as $\mathrm{Supp}(\det P)$ in all regimes considered here. The expansion and Bareiss estimates should accordingly be read as heuristics relative to this convention.

I.1 Expansion

A baseline model is expansion by minors [37,47]. In the notation used in the code and plots, this is summarized by

$$\mathcal{T}^{\mathrm{minor}} = \widetilde{O}(m 2^m \mu U).$$

The corresponding storage proxy is $\widetilde{O}(\binom{m}{m/2}U)$, since one keeps a full layer of minors rather than only the final determinant. This model is only competitive for very small matrices, but it is useful as a sanity check and as a lower-overhead baseline in the plots.

I.2 Bareiss

The fraction-free Bareiss elimination method [12] replaces cofactors by exact divisions inside a Gaussian-elimination style process. For symbolic polynomial entries, we use the coarse dense surrogate

$$\mathcal{T}^{\text{Bareiss}} = \widetilde{O}(m^3U^2).$$

This captures the fact that the arithmetic count is cubic in the matrix size, while the dominant symbolic cost comes from multiplication/division on intermediate polynomials whose support is controlled, under genericity, by the structural support proxy U. The storage proxy is correspondingly $\widetilde{O}(m^2U)$.

I.3 Kronecker

Kronecker substitution $y_i \mapsto t^{c_i}$ with $c_i = \prod_{j<i}(b_j + 1)$ maps the multivariate determinant problem to a univariate one. The substitution base b_j+1 must be at least $\deg_{y_j} \det(P) + 1$ to avoid monomial collisions in the encoded determinant; Li [61] proves that under this condition the encoding is faithful, i.e. $\det(\Psi(P)) = \Psi(\det(P))$ (Theorems 5 and 7). After substitution each *entry* of P has t-degree at most δ above, so applying the HNF algorithm of [58] to the resulting univariate polynomial matrix gives

$$T^{\text{HNF}} = \widetilde{O}(m^\omega \delta).$$

For the univariate final determinant ($k = 1$), this becomes the standard bound $\widetilde{O}\Big(\big(C_n^{(d)}\big)^\omega E_4\Big)$.

Correctness of Kronecker+HNF. Li [61] proves that Kronecker substitution commutes with the determinant (Theorems 5 and 7): if the substitution base c_i is chosen so that no monomial collisions can occur at the level of $\det(P)$, then

$$\Psi(\det(P)) = \det(\Psi(P)) \quad \text{and} \quad \det(P) = \Psi^{-1}\big(\det(\Psi(P))\big),$$

where Ψ denotes the Kronecker encoding and Ψ^{-1} its inverse. Since $\det(\Psi(P))$ is a uniquely defined univariate polynomial, *any* correct univariate determinant algorithm—including HNF [58]—produces the same output, regardless of which internal subroutines (kernel, column bases, etc.) it employs. We apply Ψ^{-1} only to the *final* univariate output, so the intermediate operations of HNF are immaterial to correctness. We have validated this experimentally on a wide range of mixed-degree systems: the Kronecker+HNF pipeline always produced the same determinant as expansion, Bareiss, and evaluation-interpolation.

I.4 Interpolation

Dense evaluation-interpolation [47] evaluates the determinant on the full tensor grid of size N and then reconstructs the polynomial variable by variable. The resulting model is

$$\mathcal{T}^{\text{eval}} = \widetilde{O}\left(NL + N\sum_{i=1}^{t}\widetilde{M}(b_i)\right) \subseteq \widetilde{O}(N(L+vB))\,.$$

The first term is the probe phase; the second is the tensor/Lagrange reconstruction phase. In the main text we use the simpler upper bound $\widetilde{O}(N(L+vB))$. This method is deterministic but becomes impractical as soon as the product grid N grows too quickly.

I.5 Sparse

When the determinant is sparse, derivative-based sparse interpolation over finite fields can be significantly better. We use the soft-Oh model inspired by Huang [48]:

$$\mathcal{T}^{\text{sparse}} = \widetilde{O}(LS + S\log q).$$

This is the *field-operation* count (intermediate step of [48, Theorem 12], before the final conversion to bit complexity); we use it because Steps 1–4 of the Dixon pipeline are uniformly counted in field operations over $\mathbb{F}_q$. The algorithm requires $\mathrm{char}(\mathbb{F}_q) \geq B$, B being a partial-degree bound on the polynomial to be interpolated (Theorem 1 of [48]); this precondition is satisfied by the large-prime fields used in Poseidon and `XHash12` but *fails* for $\mathbb{F}_{2^\ell}$ as used by `Vision`, so the Sparse model is omitted from the `Vision` analysis. The $\log q$ factor is inherent to the algorithm: the term-locator polynomial of degree $\leq S$ is recovered by equal-degree factorization, which is a Las Vegas procedure of expected cost $\widetilde{O}(S\log q)$ field operations over $\mathbb{F}_q$ (Huang [48], Theorem 12). Unlike the $\log q$ factor that appears when converting field operations to bit operations, this one is already present in the field-operation count and is not negligible for the large prime or extension fields typical of AO primitives (e.g. $q \approx 2^{64}$). The crucial input is the term bound S. In the first-determinant model we use

$$S_1 \leq \left(C_n^{(d)}\right)^2 \binom{B_1+k}{k},$$

where $C_n^{(d)}$ is the Dixon-matrix size and $B_1 = nd$ bounds the total parameter degree. This bound is structural: the bilinear Dixon representation $\Delta(\boldsymbol{x},\bar{\boldsymbol{x}}) = \sum X_u M_{u,v} \bar{X}_v$ has at most $\left(C_n^{(d)}\right)^2$ coefficient positions (the supports in $\boldsymbol{x}$ and in $\bar{\boldsymbol{x}}$ are symmetric, each of size at most $C_n^{(d)}$ by Lemma 4), and each coefficient is a parameter polynomial of total degree at most B_1, hence carries at most $\binom{B_1+k}{k}$ monomials. For the final determinant, a dense monomial upper bound gives

$$S_4 \leq \binom{D_4+k}{k}.$$

This is the key reason why sparse interpolation is the only first-determinant model that consistently stays below the final determinant in the high-variable/low-degree regime.

Stage-specific instantiations. For the first determinant we substitute $m = n$, $(\delta, N, S, L) = (\delta_1, N_1, S_1, L_1)$, and use $U = S_1$ as the intermediate-support proxy. For the final determinant we substitute $m = C_n^{(d)}$, $(\delta, N, S, L) = (\delta_4, N_4, S_4, L_4)$, and use $U = S_4$. This is exactly the convention used in the comparison plots and in the complexity-reporting code.

Where the parameters come from. The Kronecker bound δ_1 is read from the cancellation-matrix structure. In the uniform-degree case, a row-by-row analysis of the refined matrix $\widehat{C}$ gives

$$\deg_{x_i}(\det \widehat{C}) \le (i+1)d - 1, \qquad \deg_{\bar{x}_i}(\det \widehat{C}) \le (n-i)d - 1,$$

and the parameter variables satisfy $\deg_{a_j} \le d$ per row, so the total parameter degree of the $n \times n$ determinant is at most $B_1 = nd$. (See Appendix H for the derivation of these bounds.) These are exactly the ingredients used to form δ_1 and N_1 in the code. The sparse term bound S_1 comes from the Dixon-matrix construction itself: $\left(C_n^{(d)}\right)^2$ reflects the bilinear support size, while $\binom{B_1+k}{k}$ bounds the parameter contribution at each coefficient position.

For the final determinant, the HNF parameter δ_4 is obtained from the entry-wise parameter degree bound E_4, while N_4 and S_4 come from the total-degree bound D_4 of the elimination polynomial. In the well-determined case $k = 1$, this collapses to the univariate HNF model used in the main text.

Recursive block construction. Besides the five determinant models in Table 2, the code also keeps a Zhao/Fu-style recursive block-construction surrogate [82,66] that builds the Dixon matrix directly, bypassing the $n \times n$ cancellation determinant of Step 1:

$$T_2^{\mathrm{FDixon}} = \widetilde{O}\left(m_1^2 \, (n!)^3 \prod_{i=2}^{n} m_i^3 \right).$$

We do not list it in the main table because it is not a generic determinant model; rather, it is a special construction path that replaces the combined Step 1/Step 2 cost and can be attractive when the number of eliminated variables is small and the degrees are high.

J Poseidon

Poseidon [41] is a family of hash functions designed for efficient evaluation in zero-knowledge proof systems, while Poseidon2 [42] updates the linear layer for improved performance.

Let $p > 2^{30}$ be a prime and let $2 \leq t \leq 24$. Both Poseidon and Poseidon2 follow the HADES strategy. The permutation over $\mathbb{F}_p^t$ is

$$\mathcal{P}(x) = (\mathcal{E}_{R_F-1} \circ \cdots \circ \mathcal{E}_{R_F/2}) \circ (\mathcal{I}_{R_P-1} \circ \cdots \circ \mathcal{I}_0) \circ (\mathcal{E}_{R_F/2-1} \circ \cdots \circ \mathcal{E}_0) \circ M'(x),$$

where $\mathcal{E}$ denotes full rounds, $\mathcal{I}$ denotes partial rounds, and $R_F = 2R_f$. For Poseidon, M' is the identity.

The round functions are

$$\mathcal{E}_i(x) = M_E \cdot \big((x_0 + c_0^{(i)})^\alpha, \ldots, (x_{t-1} + c_{t-1}^{(i)})^\alpha\big),$$
$$\mathcal{I}_i(x) = M_I \cdot \big((x_0 + c_0^{(i)})^\alpha, x_1, \ldots, x_{t-1}\big).$$

The parameters are chosen so that $\gcd(\alpha, p-1) = 1$ and so as to meet the targeted security level.

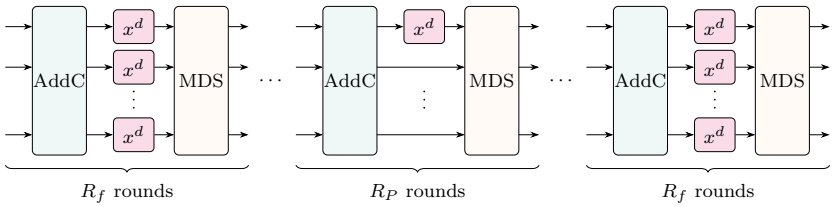


Fig. 9. Overview of Poseidon and Poseidon2. (Figure from [23])

Attack models used in Section 5. We follow the algebraic models from [43]. The direct Dixon method is applied to the input/output model, while the mixed-degree subspace models are estimated using the Hessenberg bound from Corollary 1(3). This is the reason the Poseidon part of the main text only needs the final complexity plots and timing table.

K Vision

`Vision` is a ZK-friendly hash function following the *MARVELlous* design strategy proposed in [4]. Over $\mathbb{F}_{2^\ell}$, each round alternates inverse maps with affine layers. For s branches, the round functions are

$$\mathcal{F}_{2i}(x) = \mathsf{M} \cdot \left(B^{-1}(x_0^{-1}), \ldots, B^{-1}(x_{s-1}^{-1})\right) + C_{2i},$$
$$\mathcal{F}_{2i+1}(x) = \mathsf{M} \cdot \left(B(x_0^{-1}), \ldots, B(x_{s-1}^{-1})\right) + C_{2i+1},$$

where M is an MDS matrix with rows indexed by $1, \ldots, s$, the C_j are round constants, and $B(x) = b_0 + b_1 x + b_2 x^2 + b_3 x^4$ is an $\mathbb{F}_2$-affine linearized polynomial.

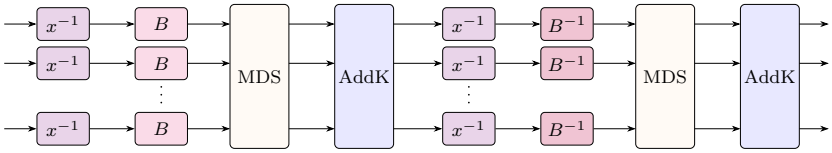


Fig. 10. Round function of `Vision`. (Figure from [23])

MITM model used in Section 5. For the CICO problem with $s = 2$, we follow the general MITM philosophy of [62] but use Dixon elimination instead of an overdefined-system Gröbner-basis attack. The detailed intermediate variables and equations are listed below because they are longer than the high-level discussion needed in the main text.

For r rounds, we introduce intermediate variables $(w_{i,1}, w_{i,2})$ and $(z_{i,1}, z_{i,2})$ at round i $(1 \le i \le r)$, so that the round-1 variables $w_{1,j}$ and $z_{1,1}$ occurring in the input constraint below are covered. The core equations are:

$$F_{i,k}^{(1)} := B(w_{i,k}) \cdot \left(\sum_{j=1}^{s} \mathsf{M}[k][j] \cdot B(z_{i-1,j}) + \epsilon_{i-1,k} \right) - 1, \qquad k \in \{1, 2\}, \quad (43)$$

for the first inverse layer, and

$$F_{i,k}^{(2)} := z_{i,k} \cdot \left(\sum_{j=1}^{s} \mathsf{M}[k][j] \cdot w_{i,j} + \sigma_{i,k} \right) - 1, \qquad k \in \{1, 2\}, \quad (44)$$

for the second inverse layer; $\epsilon_{i,k}$, $\sigma_{i,k}$ and $\beta_{3,1}$ denote the known constants of the corresponding affine layers.

The fixed-input and fixed-output constraints are represented by

$$f^{(\text{in})} := z_{1,1} \cdot \left(\sum_{j=1}^{s} \mathsf{M}[1][j] \cdot w_{1,j} + \sigma_{1,1} + \beta_{3,1} \right) - 1, \quad (45)$$

and

$$f^{(\mathrm{out})} := \sum_{j=1}^{s} \mathsf{M}[1][j] \cdot B(z_{r,j}) + \epsilon_{r,1}. \tag{46}$$

Imposing the CICO conditions fixes one input and one output coordinate and removes the corresponding pair of intermediate variables together with the two equations attached to it. What remains is one free input variable together with $2r-1$ pairs of intermediate variables, i.e. $4r-1$ variables, and the equations

$$\underbrace{2r-2}_{\deg=8,\ \text{shape } F^{(1)}} + \underbrace{2r}_{\deg=2,\ \text{shape } F^{(2)},\ \text{including } f^{(\mathrm{in})}} + \underbrace{1}_{\deg=4,\ f^{(\mathrm{out})}} = 4r-1,$$

which is the count split into the forward and backward chains in Table 10. For $r = 2$, for instance, this is 2 equations of degree 8, 4 of degree 2 and 1 of degree 4, matching the instance used in our experiments. This is the shape of the ladder (22): $|\mathcal{F}_1| = |\mathcal{X}_0| + 1 = 2$, $|\mathcal{F}_i| = 2$ for the remaining rungs, plus the single output constraint.

The equation-degree distribution used in the MITM estimate is summarized in Table 10.

Table 10. Degree distribution of equations for MITM elimination of `Vision`. Rows split each round count into the forward and backward chains; the three middle columns give the number of equations of degree 8, 2, and 4 in that chain; the last column reports the total degree of the resultant produced by that chain.

r	Equation	# Equations			Degree
		deg = 8	deg = 2	deg = 4	
even	Forward	r	r	0	2^{4r}
	Backward	$r-2$	r	1	2^{4r-4}
odd	Forward	$r-1$	$r+1$	0	2^{4r-2}
	Backward	$r-1$	$r-1$	1	2^{4r-2}

The MITM elimination proceeds by computing forward resultants up to the middle state and backward resultants from the output side to the same middle state. The two shared middle variables $\mathcal{Z}$ are retained throughout, so the forward chain terminates in one relation in $\mathcal{Z}$ of total degree D_{fwd} and the backward chain in another of total degree D_{bwd}. The final step is a single resultant within $\mathcal{Z}$: it eliminates one of the two middle variables between these two relations, yielding the univariate eliminant in the remaining middle variable. This is the detailed model underlying Equation (27) in the main text.

Rank and extraction cost of a rung. Let the incoming relation R have degree at most D in the eliminated block $\boldsymbol{x} = (x_1, x_2)$, and let the two local equations have block degree at most e. Expanding the refined cancellation determinant along

the column of R gives $\Delta = \sum_{i=1}^{3} (-1)^{i+1} \widehat{C}_{i,R} m_i$, where m_i is the corresponding local 2×2 minor. The three R-entries decompose into at most $1, D, D$ products of a polynomial in $\boldsymbol{x}$ and a polynomial in $\bar{\boldsymbol{x}}$, respectively, while grouping the local minors by dual monomials gives ranks at most $e(3e-1)/2$, $\binom{e+1}{2}$ and e, respectively. Hence

$$\operatorname{rank} \mathcal{M} \le r_0 := \frac{e(e+3)}{2} D + \frac{e(3e-1)}{2}. \tag{47}$$

The same expansion has $\mathcal{O}(e^4 D^3)$ terms and explicitly gives $\mathcal{M} = UV$ of inner dimension r_0, independently of the characteristic. After a rank-preserving specialization, a row basis $U_{I,:}$ gives $\mathcal{M}_{I,:} = U_{I,:} V$ with the same row space as $\mathcal{M}$. Thin-matrix elimination therefore extracts a maximal-rank submatrix in $\widetilde{O}(N \bar{\rho}^{\omega-1})$ operations, where $\bar{\rho} = \min\{N, r_0\}$ and N bounds both support dimensions; use dense elimination if $r_0 > N$. For fixed e, the $\widetilde{O}(D^3)$ factor-construction cost is included in Step 1.

For `Vision` with $s = 2$ the two local equations of a rung have block bidegree (e, e) with $e = 1$ for the deg $= 2$ rungs and $e = 4$ for the deg $= 8$ rungs, so (47) reads $\rho \le 2D_{\text{in}} + 1$ and $\rho \le 14 D_{\text{in}} + 22$ respectively, against a support dimension $D(\mathcal{F}) = \Theta(D_{\text{in}}^2)$.

Stage-by-stage results. Tables 11 and 12 record the cost of every elimination carried out by Method 2, so that the claim of Section 5.2 that the final merge dominates can be read off directly. For a rung we list the degree d of the local block, the total degrees D_{in} and D_{out} of the incoming and outgoing relations, the support dimension $D(\mathcal{F})$ of the associated Dixon matrix, the rank bound $\bar{\rho} = \min\{D(\mathcal{F}), r_0\}$ used for Step 3, and the costs of Steps 1, 3 and 4; the cost of the rung is their maximum. Forward rungs are numbered Fi and backward rungs Bi according to their position in the ladder. All entries are $\log_2$ field operations at $\omega = 2$. Step 1 uses the $\mathcal{O}(e^4 D_{\text{in}}^3)$ term count above, with fixed-e constants suppressed, which is far below the $\Theta(D(\mathcal{F})^2)$ coefficient positions of a dense Dixon polynomial. For Step 3 we use the rank-sensitive cost model $\widetilde{\mathcal{O}}(D(\mathcal{F}) \bar{\rho}^{\omega-1})$; its estimate remains below Step 4 for the listed parameters. The first rung has no incoming relation R and uses the ordinary two-polynomial bound $N = \bar{\rho} = 2$ instead.

The candidate roots of the final univariate eliminant are substituted back into the original system, which removes the spurious ones at negligible additional cost.

Table 11. Every Dixon elimination performed by Method 2 on `Vision` with $r = 4$ ($s = 2$), in $\log_2$ field operations, $\omega = 2$. The meet-in-the-middle cut is after rung 4.

Stage	d	$\log_2 D_{\text{in}}$	$\log_2 D_{\text{out}}$	$\log_2 D(\mathcal{F})$	$\log_2 \bar{\rho}$	Step 1	Step 3	Step 4	Cost
F1	2	0	2	1.0	1.0	3.0	2.0	5.3	5.3
F2	8	2	8	4.5	4.5	6.0	8.9	19.4	19.4
F3	2	8	10	15.0	9.0	24.0	24.0	29.0	29.0
F4	8	10	16	19.0	13.8	30.0	32.8	46.6	46.6
B7	2	2	4	3.3	3.2	6.0	6.5	11.4	11.4
B6	8	4	10	7.5	7.5	12.0	15.0	28.0	28.0
B5	2	10	12	19.0	11.0	30.0	30.0	35.0	35.0
Merge	—	16	—	16.0	16.0	48.0	32.0	48.1	**48.1**

Table 12. Maximal forward and backward stage cost against the final merge, for `Vision` with $s = 2$ and $2 \le r \le 10$, in $\log_2$ field operations, $\omega = 2$. The last column is the complexity reported for Method 2.

r	$\log_2 D_{\text{fwd}}$	$\log_2 D_{\text{bwd}}$	Max. forward	Max. backward	Merge	Max. over stages
2	8	4	19.4	11.4	24.1	24.1
3	10	10	29.0	28.0	31.0	31.0
4	16	12	46.6	35.0	48.1	48.1
5	18	18	53.0	52.6	55.0	55.0
6	24	20	70.6	59.0	72.1	72.1
7	26	26	77.0	76.6	79.0	79.0
8	32	28	94.6	83.0	96.1	96.1
9	34	34	101.0	100.6	103.0	103.0
10	40	36	118.6	107.0	120.1	120.1

L `XHash12`

`XHash12` is a STARK-friendly sponge hash function proposed by Ashur et al. [5]. The permutation over $\mathbb{F}_p^{12}$ is

$$\mathcal{F}(x) = M \circ \mathcal{B}_2 \circ \mathcal{I}_2 \circ \mathcal{P}_2 \circ \mathcal{B}_1 \circ \mathcal{I}_1 \circ \mathcal{P}_1 \circ \mathcal{B}_0 \circ \mathcal{I}_0 \circ \mathcal{P}_0(x),$$

where $p = 2^{64} - 2^{32} + 1$. The nonlinear layers combine power maps over $\mathbb{F}_p$ and over $\mathbb{F}_{p^3}$.

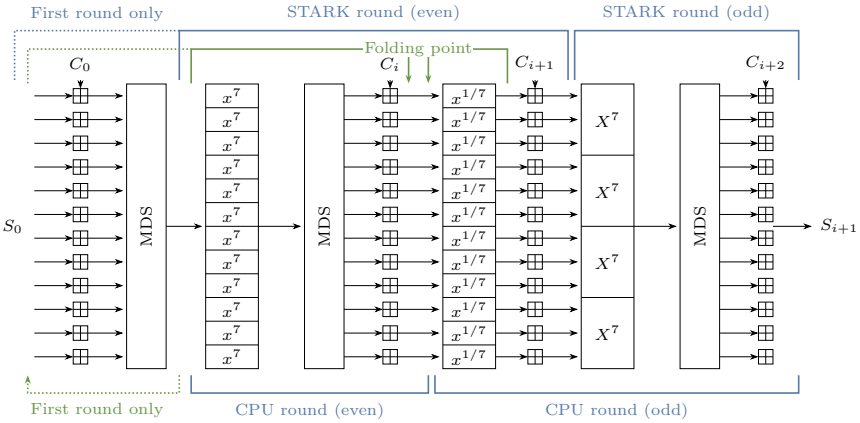


Fig. 11. Round function of `XHash12` [5]

Hybrid model used in Section 5. For an s-step instance, let $n = 12s$ and write the system in triangular form

$$\mathcal{P}_k = \begin{cases} Z_1^\alpha - f_1(\mathcal{X}) = 0 \\ \vdots \\ Z_n^\alpha - f_n(\mathcal{X}, Z_1, \ldots, Z_{n-1}) = 0 \\ h_1(\mathcal{X}, Z_1, \ldots, Z_n) = 0 \\ \vdots \\ h_k(\mathcal{X}, Z_1, \ldots, Z_n) = 0 \end{cases}$$

with $\alpha = 7$ for the standard `XHash12` instance. The complexity curves use $\bar{D} = \alpha$ for the output constraints and the degree bounds $D_i = \alpha^{n-i}\bar{D}$ before each successive elimination. For CICO-1, we instantiate (29) with $d_{\mathcal{I}} = \alpha^n$. For $k \geq 2$, we evaluate the sum in (31) and bound the final relation degrees by $\alpha^n \bar{D}$. In both the direct and final hybrid Dixon computations, construction uses the cheaper of Kronecker+HNF and cached expansion, followed by rank extraction and univariate HNF; the reported estimate is the maximum over these stages.

For CICO-1, the reduction method of [13] can be applied directly. For larger k, the hybrid strategy first derives reduced relations $R_1(\mathcal{X}), \ldots, R_k(\mathcal{X})$ from the triangular ideal and then performs a final Dixon elimination on these relations. The main text reports only the resulting complexities and timings because the qualitative conclusion is simple: the hybrid strategy is excellent for very small k, but the direct Dixon method remains the relevant estimate for the natural CICO-4 setting. The reference point for both is the direct Gröbner-basis attack of the designers [5]; the same triangular model, with the power maps of $\mathcal{B}_i$ replaced accordingly, applies verbatim to `XHash8`.